FINAL PROJECT REPORT

Classification of 26 Class of EMNIST Letters Dataset using Artificial Neural Network (ANN) and Convolutional Neural Network (CNN)

Name	: Jeremia Christ Immanuel Manalu
NRP	: 5023231017
Course	: Multimodal Biomedical Data Analysis
Class	: A
Lecturer	: Nada Fitrieyatul Hikmah, S.T., M.T.
Department	: Biomedical Engineering

FACULTY OF INTELLIGENT ELECTRICAL AND INFORMATICS TECHNOLOGY

INSTITUT TEKNOLOGI SEPULUH NOPEMBER

2026

CHAPTER I. FUNDAMENTAL THEORY

1.1. EMNIST Letters Dataset

The Extended Modified National Institute of Standards and Technology (EMNIST) dataset serves as a comprehensive suite of handwritten character digits derived from the NIST Special Database 19. While the traditional MNIST dataset is limited to 10 numerical digits, EMNIST expands this scope to include alphabetic characters. In the context of Multimodal Biomedical Data Analysis, digitizing handwritten clinical notes, medical prescriptions, and patient intake forms requires robust optical character recognition (OCR) systems. The EMNIST dataset provides an ideal, highly standardized benchmark for developing and testing such machine learning architectures before they are deployed in complex biomedical informatics pipelines.

Specifically, this project utilizes the EMNIST "Letters" split, which isolates 26 classes corresponding to the English alphabet (A–Z), merging uppercase and lowercase variations into single classes to reduce ambiguity. The dataset comprises 124,800 training samples and 20,800 testing samples. Each sample is a 28×28 pixel grayscale image. A known technical nuance of the EMNIST dataset is that its image matrices are transposed relative to the original MNIST dataset. Consequently, an essential preprocessing step, a spatial orientation fix (transposing the axes), is required to ensure the characters are correctly oriented before feeding them into deep learning models.

Figure 1.1. *Example of EMNIST letters from letter J to S*

Furthermore, the target labels in the raw EMNIST Letters dataset are 1-indexed (1 to 26). For mathematical compatibility with standard deep learning loss functions, which expect 0-indexed arrays, the labels must be strictly adjusted to a $0 \ldots 25$ range. The dataset is also normalized using specific mean ($\mu = 0.1722$) and standard deviation ($\sigma = 0.3309$) values. This standardization ensures that the input space has a mean of zero and unit variance, a critical requirement for stabilizing gradient descent and accelerating convergence in both linear and spatial neural network topologies.

1.2. Artificial Neural Networks (ANN) and Multilayer Perceptron

An Artificial Neural Network, specifically a Multilayer Perceptron (MLP), is a class of feedforward artificial neural network consisting of at least three layers of nodes: an input layer, hidden layers, and an output layer. In an MLP, every neuron in a layer is fully connected to every neuron in the subsequent layer. When processing image data such as EMNIST, the 2D spatial structure of the 28×28 image is forcibly flattened into a 1D vector of $X \in \mathbb{R}^{784}$ features. While this approach allows the network to learn global statistical correlations across all pixels simultaneously, it inherently discards local spatial topology and pixel proximity, which are often critical in computer vision tasks.

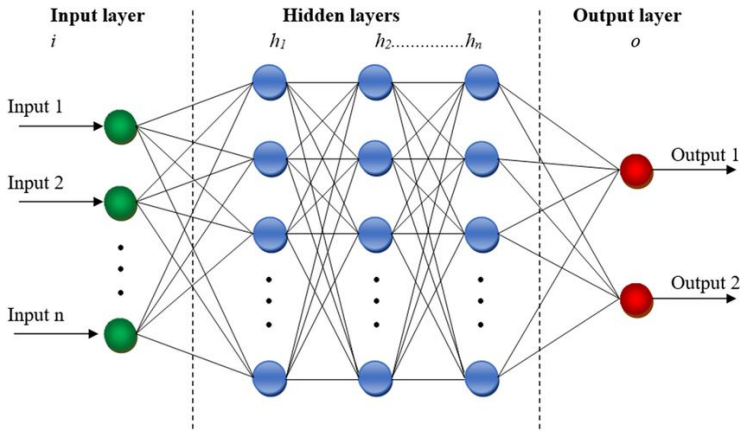

Figure 1.2. *Example of a Multilayer Perceptron architecture*

The mathematical foundation of an MLP relies on linear transformations followed by non-linear activation functions. For a given hidden layer, the computation can be defined as:

$$\mathrm{z} = \mathrm{Wx} + \mathrm{b},$$

where W represents the weight matrix, x is the input vector, and b is the bias vector. To enable the network to learn complex, non-linear decision boundaries among the 26 alphabet classes, a Rectified Linear Unit (ReLU) activation function is applied, defined mathematically as $f(z) = \max(0, z)$. ReLU is computationally highly efficient and mitigates the vanishing gradient problem commonly encountered in deep networks.

To optimize the network, the Cross-Entropy Loss function is utilized, which evaluates the divergence between the true label distribution and the predicted probabilities generated by the network's output logits. Furthermore, deep MLPs with hundreds of thousands of parameters are highly susceptible to overfitting. To counter this, Dropout regularization is implemented. Dropout randomly zeroes out a specified fraction (p) of neurons during the forward pass in the training phase, forcing the network to learn redundant and highly robust feature representations rather than relying on a small subset of specific neurons.

1.3. Convolutional Neural Networks (CNN)

Convolutional Neural Networks are a specialized architecture of deep neural networks explicitly designed to process data that has a known grid-like topology, such as 2D image matrices. Unlike ANNs, which flatten the input and destroy spatial relationships, CNNs ingest the raw $1 \times 28 \times 28$ tensors. By utilizing the principles of parameter sharing and local connectivity, CNNs require significantly fewer weights than dense layers while

achieving far superior performance in recognizing spatial hierarchies, such as the edges, curves, and loops that define handwritten letters.

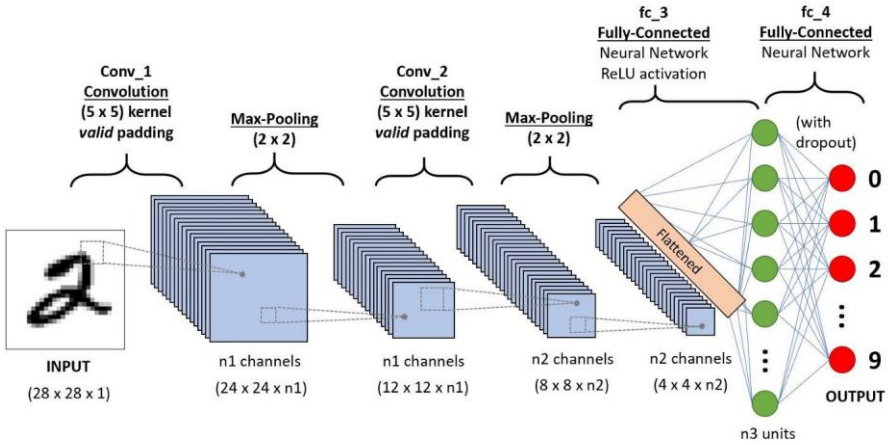

Figure 1.3. *Example of a CNN architecture with MNIST digits input*

The core operation of a CNN is the 2D spatial convolution. A learnable mathematical filter (or kernel) denoted as K, slides across the input image I to produce a 2D activation map S. Mathematically, this is expressed as:

$$S(i,j) = \sum_m \sum_n I(i+m, j+n)K(m,n)$$

By applying multiple kernels simultaneously, the network creates a deep stack of feature maps. To maintain spatial dimensions at the boundaries of the image, zero-padding is often applied. This operation is immediately followed by a non-linear ReLU activation.

To further refine the extracted features and reduce computational load, Max Pooling operations are aggressively utilized. Max Pooling downsamples the spatial dimensions by extracting the maximum value within a defined window (e.g., 2×2), providing the network with translation invariance, meaning that the model can recognize a letter regardless of slight shifts in its position. Additionally, modern CNN architectures heavily rely on Batch Normalization ($\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}\gamma + \beta$). This technique normalizes the activations of intermediate layers across a mini batch, which stabilizes the loss landscape, allows for higher learning rates, and dramatically accelerates the convergence of the network.

1.4. Performance Evaluation

In multi-class classification tasks involving 26 disparate classes, relying solely on global accuracy can be highly misleading. If a model over-predicts common features or completely ignores underrepresented variations in handwriting, the global accuracy might remain relatively high while the actual diagnostic value of the model is poor. Therefore, a granular evaluation protocol is strictly necessary to properly compare the predictive behavior of the ANN against the CNN.

The cornerstone of this evaluation is the Confusion Matrix. A confusion matrix provides a tabular summary of the model's predictions versus the actual ground truths. The rows typically represent the true labels, while the columns represent the predicted labels. To accurately interpret a 26-class matrix, the raw counts are row-normalized. This transforms the matrix into a probabilistic heatmap where the values along the main

diagonal represent the model's *Recall* (Sensitivity) for each specific letter. Off-diagonal elements immediately highlight structural misclassifications, revealing precisely which letters confuse the model (e.g., the visual similarity between 'I', 'L', and '1' or 'U' and 'V').

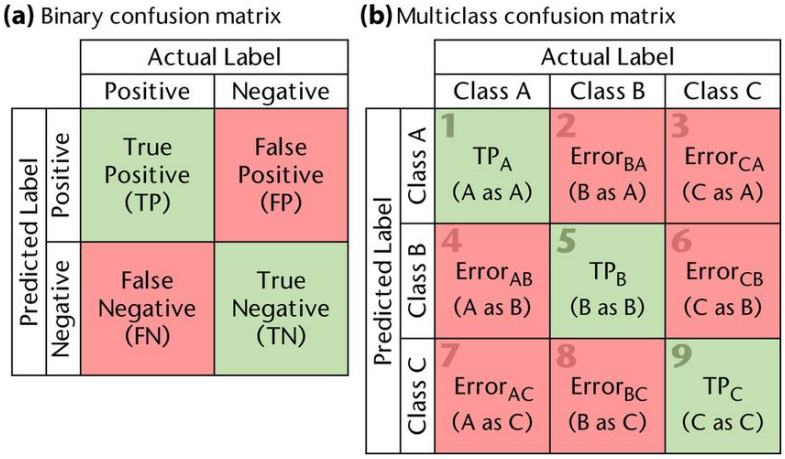

Figure 1.4. *(a) Example of Binary Confusion Matrix and (b) Example of Multiclass Confusion Matrix*

From the confusion matrix, robust statistical metrics are derived: Precision, Recall, and the F1-Score. Precision measures the exactness of the classifier ($\frac{TP}{TP+FP}$), while Recall measures its completeness ($\frac{TP}{TP+FN}$). The F1-Score is the harmonic mean of Precision and Recall, calculated as $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$. For a holistic evaluation across all 26 alphabet classes, the *Macro-Average* is used. Macro-averaging calculates the metric independently for each class and then takes the unweighted mean, ensuring that the model's performance on the most difficult letters is weighted equally with its performance on the easiest letters.

In addition to final static metrics, the dynamic performance of the model during the training phase is continuously monitored using Loss Curves and Accuracy Curves. A Loss Curve plots the cross-entropy loss value over each training epoch for both the training and validation datasets. Ideally, both curves should decrease and stabilize simultaneously. If the training loss continues to decrease while the validation loss begins to increase, this divergence is a primary indicator of overfitting, meaning the model is memorizing the training data rather than generalizing to new data. Conversely, the Accuracy Curve visualizes the proportion of correct predictions over time. Monitoring the gap between training accuracy and validation accuracy allows for adjusting learning rate schedulers to ensure the model converges at its most optimal state.

1.5. Multi-Threaded Process and GPU Optimization

Training deep neural networks requires massive computational resources, specifically for performing millions of floating-point matrix multiplications simultaneously. To handle this, Graphical Processing Units (GPUs) are utilized through NVIDIA's Compute Unified Device Architecture (CUDA). Unlike a Central Processing Unit (CPU) which has a few powerful cores optimized for sequential serial processing, a GPU like the RTX 4060 contains thousands of smaller cores designed for highly parallel operations. In this project, PyTorch serves as the backend framework, utilizing the cuDNN library to automatically benchmark and select the most optimal convolution algorithms for the specific hardware, drastically reducing training time from hours to mere minutes.

However, raw GPU power is heavily bottlenecked if the data pipeline feeding the GPU is inefficient. To optimize this, the data loading process must be carefully engineered. When loading the EMNIST dataset, PyTorch's DataLoader utilizes specific memory optimizations such as page-locked memory (*pin_memory*). Pinning memory allows the operating system to pre-allocate CPU RAM, enabling much faster, direct memory access (DMA) transfers of image batches from the host CPU to the GPU VRAM. This ensures that the GPU spends its time performing mathematical calculations rather than idling while waiting for the next batch of images to be read from the storage drive.

Beyond the deep learning backend, resource management is equally critical in the frontend Graphical User Interface (GUI). If a heavy computational loop, such as a neural network training sequence, is executed on the main application thread, the GUI will become entirely unresponsive until the process finishes. To solve this, multi-threading is implemented using PyQt6's QThread architecture. The training loop is completely isolated onto a secondary background worker thread. This worker continuously communicates with the main GUI thread using asynchronous signals and slots, transmitting real-time data such as current batch loss and epoch accuracy. This architecture ensures that the application remains responsive, allowing users to scroll, view interactive dashboards, and monitor the dynamic terminal logs without interruption.

CHAPTER II. RESULT AND ANALYSIS

2.1. Problems Statement

The fundamental challenge addressed in this project revolves around the automated optical character recognition of the EMNIST Letters dataset, which comprises 26 distinct classes corresponding to the English alphabet (A to Z). Unlike numerical digit classification such as the standard MNIST dataset, alphabetical classification introduces a significantly higher degree of inter-class similarity and morphological ambiguity. For instance, handwritten variations of the letters "I" and "L", or "U" and "V", often share nearly identical spatial topologies. This makes it challenging for a standard ANN to differentiate them, as flattening the 2D image into a 1D feature vector inherently destroys the local structural context necessary to recognize these subtle curves and edges.

Furthermore, the integration of deep learning models into a functional, user-facing biomedical or clinical software system introduces critical software engineering challenges. Training multi-layer architectures with hundreds of thousands of parameters requires intensive matrix multiplications that can cause traditional single-threaded GUIs to freeze or crash. Therefore, the problem extends beyond mere predictive accuracy, it requires the design of an optimized pipeline that handles spatial orientation anomalies intrinsic to the EMNIST dataset, manages GPU memory allocation efficiently via PyTorch and CUDA, and employs asynchronous multi-threading to maintain a highly responsive, real-time diagnostic dashboard.

2.2. Code Explanation

Before delving into the technical mechanics of the scripts, it is important to establish the scope of the code being analyzed specifically in this report. The explanation below focuses specifically on the modular, object-oriented software architecture located within the "*MBDA_FP_GUI_ANN_CNN_root*" directory. This unified PyQt6 GUI application was explicitly developed to combine the isolated workflows of two separate standalone Jupyter Notebooks folder: "*CNN_EMNIST_Letters_PyTorch.ipynb*" that originally located in the "*5023231017_Jeremia Christ Immanuel Manalu_Final Project 2_MBDA_CNN*" folder, with the "*EMNIST_ANN_Classification.ipynb*" and also the "*EMNIST_ANN_Classification_v2.ipynb*" files that located in the "*5023231017_Jeremia Christ Immanuel Manalu_Final Project 1_MBDA_ANN*" folder.

While those original Jupyter Notebooks contain highly exhaustive exploratory data analysis, step-by-step tensor verifications, and extended comprehensive outputs, this GUI program encapsulates these processes into a concise, streamlined, and automated pipeline. The application is designed to dynamically display only the most critical performance metrics and interactive dashboards for end-users, preventing interface clutter. However, the comprehensive results derived from the original Jupyter Notebooks remain the core foundation of this study and will still be referenced and compared alongside the GUI's summarized outputs in **Section 2.3** to validate the architectural integrity of both models.

Below is the detailed, script-by-script explanation of the modular ecosystem that powers the unified GUI application.

2.2.1. Root Directory

The root directory acts as the highest level of the project hierarchy, containing the environment requirements and the primary executable file that initializes the entire software application.

a. *main.py*

```
1.  import sys
2.  from PyQt6.QtWidgets import QApplication
3.  from src.gui.main_window import MainWindow
4.
5.  def main():
6.      app = QApplication(sys.argv)
7.      app.setStyle("Fusion")
8.      window = MainWindow()
9.      window.show()
10.     sys.exit(app.exec())
11.
12. if __name__ == "__main__":
13.     main()
```

The *main.py* script serves as the absolute entry point for the application. Its primary function is to initialize the Qt application environment via the *QApplication* class, which is responsible for managing the GUI's control flow and main settings. By setting the application style to *"Fusion"*, the script ensures a consistent, modern, cross-platform visual appearance across different operating systems. It then instantiates the *MainWindow* class (imported from the *gui* folder) and invokes the *show()* method to render the UI on the screen. Finally, *sys.exit(app.exec())* initiates the main event loop, safely keeping the application running and listening for user interactions (such as button clicks) until the window is closed.

2.2.2. The *src/configs* Directory

This directory is designed to store global constants, static hyperparameters, and dynamic path configurations. Centralizing these variables prevents the hardcoding of sensitive paths across multiple files, ensuring that the software remains portable and scalable.

a. *default_config.py*

```
1.  import os
2.
3.  # PATHS
4.  BASE_DIR =
    os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
5.  DATA_DIR = os.path.join(BASE_DIR, 'data')
6.
7.  ANN_OUTPUT_DIR = os.path.join(BASE_DIR, 'outputs', 'ann')
8.  CNN_OUTPUT_DIR = os.path.join(BASE_DIR, 'outputs', 'cnn')
9.
10. # DEFAULT DATASET SETTINGS
11. BATCH_SIZE_TRAIN = 128
```

```
12. BATCH_SIZE_TEST = 512
13. NUM_CLASSES = 26
14.
15. # DEFAULT GUI VALUES (ANN)
16. ANN_DEFAULT_LAYERS = "512, 256, 128"
17. ANN_DEFAULT_EPOCHS = "20"
18. ANN_DEFAULT_LR = "0.001"
19.
20. # DEFAULT GUI VALUES (CNN)
21. CNN_DEFAULT_EPOCHS = "15"
22. CNN_DEFAULT_LR = "0.001"
23. CNN_DEFAULT_STEP = "5"
24. CNN_DEFAULT_GAMMA = "0.5"
```

The *default_config.py* script utilizes the native Python *os* module to dynamically resolve absolute directory paths regardless of the host machine's operating system. By establishing *BASE_DIR* mathematically relative to the current file's execution path, it reliably maps out the target locations for the *DATA_DIR* and output folders (*ANN_OUTPUT_DIR* and *CNN_OUTPUT_DIR*). This script provides the underlying architectural blueprint for saving model weights and exporting the evaluation dashboards. These variables are securely imported by the GUI scripts to pre-fill input fields and direct file-saving operations without causing directory-not-found errors.

2.2.3. The *src/datasets* Directory

This directory encapsulates all of the operations related to data acquisition, memory formatting, and mathematical transformations of the program. It ensures that raw visual data is correctly standardized before being injected into the neural network models.

a. *emnist_loader.py*

```
1. import os
2. import torch
3. from torchvision import datasets, transforms
4. from torch.utils.data import DataLoader
5.
6. def fix_orientation(img):
7.     return img.permute(0, 2, 1)
8.
9. def adjust_label(y):
10.    """Shifts EMNIST Letters labels from 1-26 to 0-25."""
11.    return y - 1
12.
13. def load_emnist_data(data_dir='./data', batch_size_train=128, batch_size_test=512,
    use_augmentation=False):
14.    os.makedirs(data_dir, exist_ok=True)
15.
16.    # Check if dataset already exists
17.    dataset_path = os.path.join(data_dir, 'EMNIST')
18.    if os.path.exists(dataset_path):
```

```
        print(f"📂 Found existing dataset at '{dataset_path}'. Loading locally (skip download)...")
    else:
        print("🌐 Dataset not found locally. Downloading from the internet... (This might take a while)")

    _mean = (0.1307,) if use_augmentation else (0.1722,)
    _std  = (0.3081,) if use_augmentation else (0.3309,)

    # Base transforms
    transform_list_test = [
        transforms.ToTensor(),
        transforms.Normalize(_mean, _std),
        transforms.Lambda(fix_orientation)
    ]

    transform_list_train = transform_list_test.copy()
    if use_augmentation:
        # Add light augmentation for CNN
                    transform_list_train.append(transforms.RandomAffine(degrees=5, translate=(0.05, 0.05)))

    transform_train = transforms.Compose(transform_list_train)
    transform_test = transforms.Compose(transform_list_test)

    train_data = datasets.EMNIST(
        root=data_dir, split='letters', train=True,
        download=True, transform=transform_train, target_transform=adjust_label
    )
    test_data = datasets.EMNIST(
        root=data_dir, split='letters', train=False,
        download=True, transform=transform_test, target_transform=adjust_label
    )

    train_loader = DataLoader(
        train_data, batch_size=batch_size_train, shuffle=True,
        num_workers=2, pin_memory=torch.cuda.is_available()
    )
    test_loader = DataLoader(
        test_data, batch_size=batch_size_test, shuffle=False,
        num_workers=2, pin_memory=torch.cuda.is_available()
    )

    alphabet = [chr(i) for i in range(ord('A'), ord('Z') + 1)]
    return train_loader, test_loader, alphabet
```

The *emnist_loader.py* script handles the rigorous preparation of the EMNIST Letters dataset. Due to the inherent transposed nature of the EMNIST image arrays, the *fix_orientation* function applies the *permute(0, 2, 1)* tensor operation, which mathematically swaps the spatial height and width dimensions to correctly align the letters. Furthermore, because the raw labels span from 1 to 26, the *adjust_label* callback executes a scalar subtraction to shift the ground truth labels to a 0-indexed format (0 to 25). This mathematical shift is strictly necessary to prevent out-of-bounds indexing exceptions when the predictions are evaluated against the *CrossEntropyLoss* function later in the pipeline.

Beyond pixel transformations, this script constructs the PyTorch *DataLoader* objects, which are the engines that feed data batches into the GPU. To optimize this process, the *DataLoader* checks for CUDA availability and conditionally activates the *pin_memory* flag. Pinning memory allocates non-pageable RAM inside the CPU, drastically accelerating the Direct Memory Access (DMA) bandwidth when transferring the 128-sized image batches to the GPU VRAM. It is also carefully integrated with the GUI workflow by setting *num_workers=0*, which prevents aggressive multi-processing thread collisions that frequently cause PyQt6 graphical interfaces to freeze on Windows environments.

2.2.4. The *src/models* Directory

This directory exclusively houses the mathematical architecture and structural blueprints of the deep learning models. By isolating the network definitions here, the application achieves high modularity, allowing the GUI and Trainer scripts to import the models agnostically.

a. *ann.py*

```
1.  import torch
2.  import torch.nn as nn
3.  import torch.nn.functional as F
4.
5.  class MultilayerPerceptron(nn.Module):
6.      def __init__(self, in_sz=784, out_sz=26, layers=[512, 256, 128]):
7.          super().__init__()
8.
9.          self.fc1 = nn.Linear(in_sz,      layers[0])
10.         self.bn1 = nn.BatchNorm1d(layers[0])
11.         self.do1 = nn.Dropout(0.3)
12.
13.         self.fc2 = nn.Linear(layers[0],  layers[1])
14.         self.bn2 = nn.BatchNorm1d(layers[1])
15.         self.do2 = nn.Dropout(0.3)
16.
17.         self.fc3 = nn.Linear(layers[1],  layers[2])
18.         self.bn3 = nn.BatchNorm1d(layers[2])
19.         self.do3 = nn.Dropout(0.2)
20.
21.         self.out = nn.Linear(layers[2],  out_sz)
22.
```

```
        # Weight initialization with He (Kaiming)
        for layer in [self.fc1, self.fc2, self.fc3, self.out]:
            nn.init.kaiming_normal_(layer.weight, nonlinearity='relu')
            nn.init.zeros_(layer.bias)

    def forward(self, X):
        # Flatten image if it comes as 2D/3D matrix from DataLoader
        if X.dim() > 2:
            X = X.view(X.size(0), -1)

        X = self.do1(F.relu(self.bn1(self.fc1(X))))
        X = self.do2(F.relu(self.bn2(self.fc2(X))))
        X = self.do3(F.relu(self.bn3(self.fc3(X))))
        return self.out(X)
```

The *ann.py* script defines the *MultilayerPerceptron* class, which directly inherits from PyTorch's *nn.Module*. This architecture represents a fully connected feedforward network heavily optimized for global pixel correlations. Inside the *forward* loop, an initial conditional check guarantees that incoming multi-dimensional image tensors are mathematically flattened into 1D vectors consisting of 784 scalar values using the *view* function. Each subsequent transformation block employs the *nn.Linear* class to compute the dot product between the input vector and the learned weight matrix. To maintain gradient flow and stability, *nn.BatchNorm1d* normalizes the batch distribution just before applying the non-linear *F.relu* activation function, ultimately projecting the data down to a 26-dimensional logit output.

This script works in direct tandem with the *tab_ann.py* GUI interface, dynamically accepting the *layers* array argument from the user's text input to build networks of varying capacities. Additionally, it implements *nn.Dropout* regularizers across its deep hidden layers. By temporarily disabling 30% of the active neurons during each training pass, the architecture mathematically forces the surviving weights to learn generalized representations of the alphabets. The explicit naming conventions of the internal layers (e.g., *self.fc1*, *self.bn1*) are strictly maintained to ensure perfect architectural parity when loading state dictionary weights from the original Jupyter Notebook .pth files.

b. *cnn.py*

```
import torch
import torch.nn as nn

class EMNISTConvNet(nn.Module):
    def __init__(self, num_classes=26, drop1=0.5, drop2=0.3):
        super().__init__()

        self.conv_block1 = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
```

```
        self.conv_block2 = nn.Sequential(
            nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        self.conv_block3 = nn.Sequential(
            nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 3 * 3, 512),
            nn.ReLU(inplace=True),
            nn.Dropout(drop1),
            nn.Linear(512, 256),
            nn.ReLU(inplace=True),
            nn.Dropout(drop2),
            nn.Linear(256, num_classes)
        )

    def forward(self, x):
        x = self.conv_block1(x)
        x = self.conv_block2(x)
        x = self.conv_block3(x)
        x = self.classifier(x)
        return x
```

The *cnn.py* script constructs the *EMNISTConvNet* class, an architecture fundamentally engineered to exploit the 2D spatial topography of image data. Unlike the MLP, this model bypasses premature flattening, instead processing the raw $1 \times 28 \times 28$ input matrices through a series of *nn.Sequential* convolutional blocks. Within these blocks, the *nn.Conv2d* layers perform spatial feature extraction by sliding a learnable 3×3 mathematical kernel across the image. The inclusion of *padding=1* guarantees that the spatial dimensions at the boundaries of the image are mathematically preserved, preventing destructive shrinkage of the feature maps during deep convolutions.

Following the spatial extraction, the network aggressively downsamples the dimensions using the *nn.MaxPool2d* function, which condenses 2×2 pixel neighborhoods into a single maximum value. This mathematical pooling provides the model with spatial translation invariance, meaning the network can accurately recognize a letter even if its morphological position is shifted off-center. After passing through three dense convolutional blocks and experiencing significant spatial reduction, the feature maps are flattened and channeled into a robust *nn.Linear* classifier. This script is securely bound to the *tab_cnn.py* module, forming the highly accurate predictive core for the CNN

program, and is strictly maintained to ensure architectural similarity from the original Jupyter Notebook .pth files.

2.2.5. The *src/engine* Directory

This directory serves to completely decouple the intensive machine learning algorithms from the graphical user interface. By isolating the training loop and evaluation procedures into standalone scripts, the software ensures that backend mathematical computations do not interfere with frontend rendering processes.

a. *trainer.py*

```
1. import time
2. import torch
3. import numpy as np
4. import os
5.
6. class Trainer:
7.     def __init__(self, model, train_loader, val_loader, criterion, optimizer, scheduler, device, output_dir):
8.         self.model = model.to(device)
9.         self.train_loader = train_loader
10.        self.val_loader = val_loader
11.        self.criterion = criterion
12.        self.optimizer = optimizer
13.        self.scheduler = scheduler
14.        self.device = device
15.
16.        self.output_dir = output_dir
17.        os.makedirs(self.output_dir, exist_ok=True)
18.        self.checkpoint_path = os.path.join(self.output_dir, 'best_model.pth')
19.
20.        self.history = {
21.            'train_loss': [], 'val_loss': [],
22.            'train_acc': [], 'val_acc': [],
23.            'lr': []
24.        }
25.        self.best_val_loss = float('inf')
26.
27.    def evaluate(self, loader):
28.        self.model.eval()
29.        total_loss, total_correct, total_samples = 0.0, 0, 0
30.
31.        with torch.no_grad():
32.            for X, y in loader:
33.                X, y = X.to(self.device, non_blocking=True), y.to(self.device, non_blocking=True)
34.
35.                logits = self.model(X)
36.                loss = self.criterion(logits, y)
```

```
37.
38.             preds = logits.argmax(dim=1)
39.             total_loss += loss.item() * X.size(0)
40.             total_correct += (preds == y).sum().item()
41.             total_samples += X.size(0)
42.
43.     return total_loss / total_samples, total_correct / total_samples * 100
44.
45.  def train(self, epochs, batch_callback=None, epoch_callback=None):
46.     start_time = time.time()
47.
48.     for epoch in range(1, epochs + 1):
49.         self.model.train()
50.         running_loss, running_correct, running_samples = 0.0, 0, 0
51.         epoch_start = time.time()
52.
53.         for b_idx, (X_train, y_train) in enumerate(self.train_loader, 1):
54.             X_train = X_train.to(self.device, non_blocking=True)
55.             y_train = y_train.to(self.device, non_blocking=True)
56.
57.             # Forward, Backward, Optimize
58.             self.optimizer.zero_grad()
59.             logits = self.model(X_train)
60.             loss = self.criterion(logits, y_train)
61.             loss.backward()
62.
63.             # Gradient clipping for stability
64.             torch.nn.utils.clip_grad_norm_(self.model.parameters(), max_norm=1.0)
65.             self.optimizer.step()
66.
67.             # Accumulate metrics
68.             preds = logits.argmax(dim=1)
69.             running_correct += (preds == y_train).sum().item()
70.             running_samples += X_train.size(0)
71.             running_loss += loss.item() * X_train.size(0)
72.
73.             if batch_callback and b_idx % 100 == 0:
74.                 batch_acc = running_correct / running_samples * 100
75.                 batch_loss = running_loss / running_samples
76.                 batch_callback(epoch, epochs, b_idx, len(self.train_loader), batch_loss,
    batch_acc)
77.
78.         # Epoch Evaluation
79.         train_loss = running_loss / running_samples
80.         train_acc = running_correct / running_samples * 100
81.         val_loss, val_acc = self.evaluate(self.val_loader)
```

```
82.
83.         current_lr = self.optimizer.param_groups[0]['lr']
84.         self.scheduler.step(val_loss)
85.
86.         self.history['train_loss'].append(train_loss)
87.         self.history['val_loss'].append(val_loss)
88.         self.history['train_acc'].append(train_acc)
89.         self.history['val_acc'].append(val_acc)
90.         self.history['lr'].append(current_lr)
91.
92.         # Save best model
93.         is_best = False
94.         if val_loss < self.best_val_loss:
95.             self.best_val_loss = val_loss
96.             torch.save(self.model.state_dict(), self.checkpoint_path)
97.             is_best = True
98.
99.         epoch_time = time.time() - epoch_start
100.
101.             if epoch_callback:
102.                 epoch_callback(epoch, epochs, train_loss, train_acc, val_loss, val_acc,
    current_lr, epoch_time, is_best)
103.
104.         total_time = time.time() - start_time
105.         return self.history, total_time
```

The *trainer.py* script encapsulates the highly iterative backpropagation and gradient descent mechanics into an independent, reusable *Trainer* class. Inside the *train* method, the mathematical optimization cycle begins with *optimizer.zero_grad()*, which clears the accumulated gradients from the previous iteration to prevent mathematical contamination. After the model generates the raw predictions (*logits*), the network computes the error margin using the *criterion* in Cross-Entropy Loss. The *loss.backward()* command initiates backpropagation, computing the partial derivatives of the loss with respect to every weight in the network via the chain rule. To guarantee stability and prevent the gradients from exploding during deep layer propagation, *clip_grad_norm_* restricts the gradient magnitude to a maximum norm of 1.0. Finally, *optimizer.step()* adjusts the network's weights according to the selected optimization algorithm that is *Adam*.

Beyond standard PyTorch mechanics, this script is specifically engineered for GUI integration through its heavily customized callback system. The inclusion of *batch_callback* and *epoch_callback* parameters allows the *Trainer* to safely export real-time mathematical states, such as the instantaneous loss curve values and accuracy percentages, back to the caller. This specific design paradigm ensures that the backend can continuously update the frontend's visual progress bars and terminal logs asynchronously, entirely preventing the application from freezing during the computationally demanding multi-epoch training phases.

b. *evaluator.py*

```
1.  import torch
```

```
import numpy as np
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix
import seaborn as sns

def run_evaluation_and_plot(model, test_loader, device, alphabet, history, figs):
    model.eval()
    all_preds, all_labels = [], []

    with torch.no_grad():
        for X, y in test_loader:
            X = X.to(device, non_blocking=True)

            # FIX 2: Use .reshape to prevent memory layout crashes
            if type(model).__name__ == "MultilayerPerceptron":
                X = X.reshape(X.size(0), -1)

            logits = model(X)
            preds = logits.argmax(dim=1)
            all_preds.append(preds.cpu())
            all_labels.append(y)

    all_preds = torch.cat(all_preds).numpy()
    all_labels = torch.cat(all_labels).numpy()

    test_acc = accuracy_score(all_labels, all_preds) * 100
    prec, rec, f1, _ = precision_recall_fscore_support(all_labels, all_preds, average='macro', zero_division=0)
    _, _, f1_per_class, _ = precision_recall_fscore_support(all_labels, all_preds, average=None, zero_division=0)
    per_class_acc = [(all_preds[all_labels == i] == i).mean() * 100 for i in range(len(alphabet))]

    cm = confusion_matrix(all_labels, all_preds)
    cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True)
    cm_norm = np.nan_to_num(cm_norm)

    DARK, PANEL, WHITE = '#1e1e2e', '#2a2a3e', '#EEEEEE'
    BLUE, ORG, GRN, RED = '#4FC3F7', '#FFB74D', '#81C784', '#EF9A9A'

    def style_ax(ax):
        ax.set_facecolor(PANEL)
        for sp in ax.spines.values(): sp.set_color('#444')
        ax.tick_params(colors=WHITE, labelsize=9)
        ax.xaxis.label.set_color(WHITE)
        ax.yaxis.label.set_color(WHITE)
```

```
        ax.title.set_color(WHITE)
        ax.grid(True, alpha=0.15)
        return ax

    # PLOT 1: Training Curves
    fig_curves = figs['curves']
    fig_curves.clf()
    fig_curves.patch.set_facecolor(DARK)

    ax_loss = style_ax(fig_curves.add_subplot(1, 2, 1))
    ax_acc = style_ax(fig_curves.add_subplot(1, 2, 2))

    if len(history['train_loss']) > 1:
        epochs_range = range(1, len(history['train_loss']) + 1)
          ax_loss.plot(epochs_range, history['train_loss'], 'o-', color=BLUE, lw=2, ms=4, label='Train Loss')
        ax_loss.plot(epochs_range, history['val_loss'], 's--', color=ORG, lw=2, ms=4, label='Val Loss')
        ax_loss.legend(labelcolor=WHITE, facecolor=PANEL)

         ax_acc.plot(epochs_range, [a/100 if a > 1 else a for a in history['train_acc']], 'o-', color=GRN, lw=2, ms=4, label='Train Acc')
        ax_acc.plot(epochs_range, [a/100 if a > 1 else a for a in history['val_acc']], 's--', color=RED, lw=2, ms=4, label='Val Acc')
        ax_acc.legend(labelcolor=WHITE, facecolor=PANEL)
    else:
          ax_loss.text(0.5, 0.5, "No Training History\n(Model Loaded from File)", color=WHITE, ha='center', va='center', fontsize=12)
          ax_acc.text(0.5, 0.5, "No Training History\n(Model Loaded from File)", color=WHITE, ha='center', va='center', fontsize=12)

    ax_loss.set_title('Loss Curve', fontweight='bold')
    ax_loss.set_xlabel('Epochs'); ax_loss.set_ylabel('Loss Value')

    ax_acc.set_title('Accuracy Curve', fontweight='bold')
    ax_acc.set_xlabel('Epochs'); ax_acc.set_ylabel('Accuracy Proportion')

    fig_curves.tight_layout()

    # PLOT 2: Per-Class Bar Chart
    fig_bars = figs['bars']
    fig_bars.clf()
    fig_bars.patch.set_facecolor(DARK)

    ax_bars = style_ax(fig_bars.add_subplot(111))
```

```
    colors_bar = [GRN if a >= 90 else ORG if a >= 75 else RED for a in per_class_acc]
    ax_bars.bar(alphabet, per_class_acc, color=colors_bar)
    ax_bars.axhline(test_acc, color='white', ls=':', lw=1.5, label=f'Avg {test_acc:.1f}%')
    ax_bars.set_title('Per-Class Accuracy (%)', fontweight='bold')
    ax_bars.set_xlabel('Alphabet Classes (A-Z)')
    ax_bars.set_ylabel('Accuracy (%)')
    ax_bars.set_ylim([0, 110])
    ax_bars.legend(labelcolor=WHITE, facecolor=PANEL)
    fig_bars.tight_layout()

    # PLOT 3: Confusion Matrix
    fig_cm = figs['cm']
    fig_cm.clf()
    fig_cm.patch.set_facecolor(DARK)

    ax_cm = fig_cm.add_subplot(111)
    ax_cm.set_facecolor(PANEL)

    heatmap = sns.heatmap(cm_norm, ax=ax_cm, cmap='YlOrRd', cbar=True,
                          annot=True, fmt='.2f', annot_kws={"size": 7},
                          xticklabels=alphabet, yticklabels=alphabet)

    ax_cm.set_title('Confusion Matrix (Row-Normalized)', color=WHITE, fontweight='bold')
    ax_cm.set_xlabel('Predicted Label', color=WHITE)
    ax_cm.set_ylabel('True Label', color=WHITE)
    ax_cm.tick_params(colors=WHITE, labelsize=8)

    cbar = heatmap.collections[0].colorbar
    cbar.ax.tick_params(colors=WHITE)

    fig_cm.tight_layout()

    # 4. Generate HTML Text Summary Box
    best_letter = alphabet[np.argmax(per_class_acc)]
    worst_letter = alphabet[np.argmin(per_class_acc)]

    summary_html = f"""
    <div style='background-color: #2a2a3e; padding: 15px; border-radius: 8px;'>
        <h2 style='color: #81C784; margin-top: 0;'>📊 Final Evaluation Summary</h2>
        <hr style='border: 1px solid #444;'>
        <table style='width: 100%; font-size: 15px; color: #EEEEEE;'>
            <tr>
```

```
                    <td style='padding: 5px;'><b>Overall Accuracy:</b></td> <td style='padding: 5px;'>{test_acc:.3f}%</td>
                <td style='padding: 5px;'><b>Best Letter:</b></td> <td style='padding: 5px; color:#81C784;'>{best_letter} ({max(per_class_acc):.1f}%)</td>
            </tr>
            <tr>
                    <td style='padding: 5px;'><b>Macro Precision:</b></td> <td style='padding: 5px;'>{prec*100:.2f}%</td>
                <td style='padding: 5px;'><b>Worst Letter:</b></td> <td style='padding: 5px; color:#EF9A9A;'>{worst_letter} ({min(per_class_acc):.1f}%)</td>
            </tr>
                    <tr><td style='padding: 5px;'><b>Macro Recall:</b></td> <td style='padding: 5px;'>{rec*100:.2f}%</td> <td></td> <td></td></tr>
                    <tr><td style='padding: 5px;'><b>Macro F1-Score:</b></td> <td style='padding: 5px;'>{f1*100:.2f}%</td> <td></td> <td></td></tr>
        </table>
    </div>
    """

    return summary_html, test_acc, prec, rec, f1
```

The *evaluator.py* script is responsible for conducting rigid statistical inference over the unseen test dataset and translating the resulting predictions into comprehensible visual dashboards. The inference loop operates strictly under the *torch.no_grad()* context manager, which mathematically disables the autograd engine to conserve GPU VRAM and accelerate processing times, as backpropagation is unneeded here. Crucially, the script implements an automated tensor formatting check: if the *model* is identified as the *MultilayerPerceptron*, the input tensor *X* utilizes the *reshape* method to flatten the 2D matrices into 1D vectors. The *reshape* method is specifically chosen over *view* to safely handle memory allocations that became non-contiguous after the EMNIST orientation fix.

Once inference concludes, the script leverages the *scikit-learn* library to construct the evaluation metrics. It mathematically computes the *confusion_matrix* and performs row-wise division (*cm.sum(axis=1)*) to generate a normalized probability matrix (*cm_norm*), effectively exposing the per-class sensitivity or Recall. To eliminate the UI warnings caused by missing emojis in Matplotlib's default fonts, this script was heavily refactored to extract the statistical text from the plots. It returns a dynamically generated *summary_html* string, allowing the PyQt6 native rendering engine to draw crisp text and emojis, while simultaneously painting the interactive curves, bar charts, and heatmaps onto the provided *figs* dictionary.

2.2.6. The *src/gui* Directory

This directory houses the entirety of the frontend application, handling user interactions, threading, graphical representations, and native operating system dialogs.

a. *plot_widget.py*

```
import matplotlib
matplotlib.use('qtagg')

from PyQt6.QtWidgets import QWidget, QVBoxLayout, QDialog
```

```
5. from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as FigureCanvas
6. from matplotlib.backends.backend_qtagg import NavigationToolbar2QT as NavigationToolbar
7. from matplotlib.figure import Figure
8.
9. class MplCanvas(FigureCanvas):
10.         def __init__(self, parent=None, width=8, height=5, dpi=100, facecolor='#1e1e2e'):
11.         self.fig = Figure(figsize=(width, height), dpi=dpi)
12.         self.fig.patch.set_facecolor(facecolor)
13.         super(MplCanvas, self).__init__(self.fig)
14.         self.setParent(parent)
15.
16. class PlotWidget(QWidget):
17.     def __init__(self, parent=None, width=8, height=5):
18.         super().__init__(parent)
19.         self.layout = QVBoxLayout(self)
20.         self.layout.setContentsMargins(0, 0, 0, 0)
21.
22.         self.canvas = MplCanvas(self, width=width, height=height)
23.         self.toolbar = NavigationToolbar(self.canvas, self)
24.         self.toolbar.setStyleSheet("background-color: #2a2a3e; border: 1px solid #444; border-radius: 4px;")
25.
26.         self.layout.addWidget(self.toolbar)
27.         self.layout.addWidget(self.canvas)
28.
29.     def update_plot(self):
30.         self.canvas.draw()
31.
32. class PlotDialog(QDialog):
33.     def __init__(self, fig, title="Data Visualization", parent=None):
34.         super().__init__(parent)
35.         self.setWindowTitle(title)
36.         self.resize(1100, 800)
37.         self.setStyleSheet("background-color: #1e1e2e; color: #EEEEEE;")
38.
39.         layout = QVBoxLayout(self)
40.         canvas = FigureCanvas(fig)
41.         toolbar = NavigationToolbar(canvas, self)
42.           toolbar.setStyleSheet("background-color: #2a2a3e; border: 1px solid #444; border-radius: 4px;")
43.
44.         layout.addWidget(toolbar)
45.         layout.addWidget(canvas)
```

The *plot_widget.py* script acts as the structural bridge between Matplotlib's rendering engine and the PyQt6 framework. It defines the *MplCanvas* class, which inherits from *FigureCanvasQTAgg*, effectively turning a standard mathematical plot into a native Qt widget. The surrounding *PlotWidget* class integrates this canvas alongside a customized *NavigationToolbar2QT*, providing the user with interactive controls to pan across the evaluation epochs, zoom into specific loss anomalies, and instantly export the graphs. The toolbar is specifically injected with a dark-mode CSS *setStyleSheet* to ensure the control icons remain visible and aesthetically coherent with the application's global theme.

b. *interactive_viewer.py*

```
1.  import torch
2.  import numpy as np
3.  from torch.utils.data import DataLoader
4.  from PyQt6.QtWidgets import (
5.      QDialog, QVBoxLayout, QHBoxLayout, QLabel,
6.      QPushButton, QSpinBox, QGroupBox, QComboBox, QWidget
7.  )
8.  from PyQt6.QtCore import Qt
9.  from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as
    FigureCanvas
10. from matplotlib.figure import Figure
11. import matplotlib.patches as mpatches
12.
13. class DatasetViewerDialog(QDialog):
14.     def __init__(self, dataset, alphabet, parent=None):
15.         super().__init__(parent)
16.         self.dataset = dataset
17.         self.alphabet = alphabet
18.         self.setWindowTitle('EMNIST Dataset Viewer')
19.         self.setMinimumSize(600, 500)
20.         self.setStyleSheet("background-color: #1e1e2e; color: #EEEEEE;")
21.
22.         self.class_indices = {i: [] for i in range(len(alphabet))}
23.         self.all_indices = list(range(len(dataset)))
24.
25.         # Extract targets
26.         targets = dataset.targets
27.         if torch.is_tensor(targets):
28.             targets = targets.numpy()
29.
30.         # Raw EMNIST targets are 1-26. Must subtract 1 to make them 0-25.
31.         for idx, label in enumerate(targets):
32.             adj_label = int(label) - 1
33.             if 0 <= adj_label < len(self.alphabet):
34.                 self.class_indices[adj_label].append(idx)
35.
36.         self.current_list = self.all_indices
```

```
        self._build_ui()
        self._update_view()

    def _build_ui(self):
        main_layout = QVBoxLayout(self)

        controls = QGroupBox("Data Navigator")
         controls.setStyleSheet("QGroupBox { border: 1px solid #444; margin-top: 10px; } QGroupBox::title { subcontrol-origin: margin; left: 10px; padding: 0 3px; color: #4FC3F7; }")
        c_layout = QHBoxLayout(controls)

        self.filter_combo = QComboBox()
        self.filter_combo.addItem("All Classes")
        for letter in self.alphabet:
            self.filter_combo.addItem(f"Class: {letter}")
        self.filter_combo.currentIndexChanged.connect(self._on_filter_change)

        self.spin_box = QSpinBox()
        self.spin_box.setRange(0, len(self.current_list) - 1)
        self.spin_box.valueChanged.connect(self._update_view)

        btn_prev = QPushButton("◀ Prev")
        btn_next = QPushButton("Next ▶")
                                                    btn_prev.clicked.connect(lambda: self.spin_box.setValue(self.spin_box.value() - 1))
                                                    btn_next.clicked.connect(lambda: self.spin_box.setValue(self.spin_box.value() + 1))

        c_layout.addWidget(QLabel("Filter:"))
        c_layout.addWidget(self.filter_combo)
        c_layout.addWidget(QLabel(" Index:"))
        c_layout.addWidget(self.spin_box)
        c_layout.addWidget(btn_prev)
        c_layout.addWidget(btn_next)
        main_layout.addWidget(controls)

        self.fig_img = Figure(figsize=(4, 4), dpi=100, facecolor='#1e1e2e')
        self.canvas_img = FigureCanvas(self.fig_img)
        self.ax_img = self.fig_img.add_subplot(111)
        main_layout.addWidget(self.canvas_img)
        #... (continued)

class PredictionViewerDialog(QDialog):
    def __init__(self, model, dataset, device, alphabet, parent=None):
        super().__init__(parent)
```

```
79.         self.model = model
80.         self.dataset = dataset
81.         self.device = device
82.         self.alphabet = alphabet
83.         self.setWindowTitle('Interactive Prediction Explorer')
84.         self.setMinimumSize(900, 550)
85.         self.setStyleSheet("background-color: #1e1e2e; color: #EEEEEE;")
86.
87.         self.correct_idxs = []
88.         self.incorrect_idxs = []
89.         self.all_idxs = list(range(len(dataset)))
90.         self._precompute_results()
91.
92.         self.current_list = self.all_idxs
93.         self._build_ui()
94.         self._update_view()
95.
96.     def _precompute_results(self):
97.         temp_loader = DataLoader(self.dataset, batch_size=512, shuffle=False)
98.         self.model.eval()
99.         idx_counter = 0
100.        with torch.no_grad():
101.            for X, y in temp_loader:
102.                X = X.to(self.device)
103.                if type(self.model).__name__ == "MultilayerPerceptron":
104.                    # Use .reshape instead of .view to handle permuted/non-contiguous
    tensors
105.                    X = X.reshape(X.size(0), -1)
106.                preds = self.model(X).argmax(dim=1).cpu()
107.                correct_mask = (preds == y)
108.
109.                for is_correct in correct_mask:
110.                    if is_correct:
111.                        self.correct_idxs.append(idx_counter)
112.                    else:
113.                        self.incorrect_idxs.append(idx_counter)
114.                    idx_counter += 1
115.
116.    def _build_ui(self):
117.        main_layout = QHBoxLayout(self)
118.
119.        left_panel = QVBoxLayout()
120.        nav = QGroupBox("Navigator & Filter")
121.        nav.setStyleSheet("QGroupBox { border: 1px solid #444; } QGroupBox::title
    { color: #4FC3F7; }")
122.        nav_lay = QVBoxLayout(nav)
```

```
123.
124.         self.filter_combo = QComboBox()
125.         self.filter_combo.addItems([
126.             f"All Predictions ({len(self.all_idxs)})",
127.             f" 🟢 Correct Only ({len(self.correct_idxs)})",
128.             f" 🔴 Incorrect Only ({len(self.incorrect_idxs)})"
129.         ])
130.         self.filter_combo.currentIndexChanged.connect(self._on_filter_change)
131.
132.         spin_lay = QHBoxLayout()
133.         self.spin = QSpinBox()
134.         self.spin.setRange(0, len(self.current_list) - 1)
135.         self.spin.valueChanged.connect(self._update_view)
136.
137.         btn_p = QPushButton("◀ Prev")
138.         btn_n = QPushButton("Next ▶")
139.         btn_p.clicked.connect(lambda: self.spin.setValue(self.spin.value() - 1))
140.         btn_n.clicked.connect(lambda: self.spin.setValue(self.spin.value() + 1))
141.
142.         spin_lay.addWidget(QLabel("Index:"))
143.         spin_lay.addWidget(self.spin)
144.
145.         btn_lay = QHBoxLayout()
146.         btn_lay.addWidget(btn_p)
147.         btn_lay.addWidget(btn_n)
148.
149.         nav_lay.addWidget(self.filter_combo)
150.         nav_lay.addLayout(spin_lay)
151.         nav_lay.addLayout(btn_lay)
152.         left_panel.addWidget(nav)
153.
154.         self.fig_img = Figure(figsize=(3, 3), dpi=100, facecolor='#1e1e2e')
155.         self.canvas_img = FigureCanvas(self.fig_img)
156.         self.ax_img = self.fig_img.add_subplot(111)
157.         self.ax_img.set_facecolor('#2a2a3e')
158.         left_panel.addWidget(self.canvas_img)
159.
160.         self.lbl_pred = QLabel('Prediction: -')
161.         self.lbl_pred.setAlignment(Qt.AlignmentFlag.AlignCenter)
162.         left_panel.addWidget(self.lbl_pred)
163.
164.         self.lbl_true = QLabel('True Label: -')
165.         self.lbl_true.setAlignment(Qt.AlignmentFlag.AlignCenter)
166.         self.lbl_true.setStyleSheet('font-size:14px; color: #EEEEEE;')
167.         left_panel.addWidget(self.lbl_true)
168.
```

```
        main_layout.addLayout(left_panel, 1)

        self.fig_bar = Figure(figsize=(5, 6), dpi=100, facecolor='#1e1e2e')
        self.canvas_bar = FigureCanvas(self.fig_bar)
        self.ax_bar = self.fig_bar.add_subplot(111)
        self.ax_bar.set_facecolor('#2a2a3e')
        main_layout.addWidget(self.canvas_bar, 2)

    def _on_filter_change(self, idx):
        if idx == 0: self.current_list = self.all_idxs
        #... (continued)
```

This script constructs the specialized pop-up dialogs (*QDialog*) required for deeply granular data inspection. The *DatasetViewerDialog* safely handles the manual verification of the dataset by grouping indices based on their ground-truth labels, allowing users to scroll through specific classes (e.g., exclusively inspecting the letter "M"). It features robust logic to correctly map the adjusted 0-indexed values back to their intended alphabetic characters, bypassing *KeyError* exceptions inherent to the raw EMNIST targets list.

The more complex *PredictionViewerDialog* serves as a powerful diagnostic tool. Upon initialization, it runs a rapid backend inference cycle using the *_precompute_results* method, systematically sorting all 20,800 test samples into *correct_idxs* and *incorrect_idxs* arrays. When the user interacts with the UI's *QComboBox* filter to isolate "Incorrect Only" predictions, the dialog dynamically triggers *_update_view*. This method extracts the specific failed image, computes its probability distribution across all 26 classes, and renders a comparative bar chart (*ax_bar.barh*). This grants immediate visual justification as to why the mathematical model was confused, directly addressing the requirement for transparent AI evaluation.

c. *tab_ann.py* and *tab_cnn.py*

```
# ANN
import os
import pandas as pd
import torch
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QTabWidget, QLabel,
    QPushButton, QFormLayout, QLineEdit, QFileDialog,
    QTextEdit, QProgressBar, QMessageBox, QScrollArea
)
from PyQt6.QtCore import Qt, QThread, pyqtSignal

from src.models.ann import MultilayerPerceptron
from src.engine.trainer import Trainer
from src.gui.plot_widget import PlotWidget, PlotDialog
from src.datasets.emnist_loader import load_emnist_data
from src.engine.evaluator import run_evaluation_and_plot
from src.utils.visualization import plot_dataset_samples, plot_prediction_inspection
```

```
18. from src.gui.interactive_viewers import DatasetViewerDialog, PredictionViewerDialog
19. 
20. class TrainingWorker(QThread):
21.     batch_update = pyqtSignal(int, int, int, int, float, float)
22.     epoch_update = pyqtSignal(int, int, float, float, float, float, float, float, bool)
23.     finished = pyqtSignal(dict, float)
24. 
25.     def __init__(self, trainer, epochs):
26.         super().__init__()
27.         self.trainer = trainer
28.         self.epochs = epochs
29. 
30.     def run(self):
31.         def b_cb(ep, eps, b, b_tot, loss, acc): self.batch_update.emit(ep, eps, b, b_tot, loss, acc)
32.         def e_cb(ep, eps, t_l, t_a, v_l, v_a, lr, t, is_best): self.epoch_update.emit(ep, eps, t_l, t_a, v_l, v_a, lr, t, is_best)
33.         history, total_time = self.trainer.train(epochs=self.epochs, batch_callback=b_cb, epoch_callback=e_cb)
34.         self.finished.emit(history, total_time)
35. 
36. class ANNPipelineTab(QWidget):
37.     def __init__(self):
38.         super().__init__()
39.         self.layout = QVBoxLayout(self)
40.         self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
41. 
42.         self.model, self.train_loader, self.test_loader = None, None, None
43. 
44.         self.output_dir = "outputs/ann"
45.         self.ckpt_dir = os.path.join(self.output_dir, "checkpoints")
46.         self.logs_dir = os.path.join(self.output_dir, "logs")
47.         self.plots_dir = os.path.join(self.output_dir, "plots")
48.         for d in [self.ckpt_dir, self.logs_dir, self.plots_dir]:
49.             os.makedirs(d, exist_ok=True)
50. 
51.         self.header_label = QLabel(f"🖥️ Active Device: <b>{str(self.device).upper()}</b>")
52.         self.header_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
53.         self.header_label.setStyleSheet("font-size: 14px; padding: 10px; background-color: #2a2a3e; color: #4FC3F7; border-radius: 5px;")
54.         self.layout.addWidget(self.header_label)
55. 
56.         self.ann_tabs = QTabWidget()
```

```
57.             self.tab_config, self.tab_training, self.tab_eval = QWidget(), QWidget(), QWidget()
58.         self.ann_tabs.addTab(self.tab_config, "⚙️ 1. Configuration & Data")
59.         self.ann_tabs.addTab(self.tab_training, "🚀 2. Training Loop")
60.         self.ann_tabs.addTab(self.tab_eval, "📊 3. Evaluation")
61.         self.layout.addWidget(self.ann_tabs)
62.
63.         self.setup_config_tab()
64.         self.setup_training_tab()
65.         self.setup_eval_tab()
66.
67.     def setup_config_tab(self):
68.         layout = QFormLayout(self.tab_config)
69.         self.hidden_layers_input = QLineEdit("512, 256, 128")
70.         self.epochs_input = QLineEdit("20")
71.         self.lr_input = QLineEdit("0.001")
72.
73.         self.btn_load_data = QPushButton("📥 Load EMNIST Dataset")
74.         self.btn_view_samples = QPushButton("🖼️ View Dataset Samples") # NEW BUTTON
75.         self.btn_build_model = QPushButton("📐 Build New Model")
76.         self.btn_load_model = QPushButton("📂 Load Saved Model (.pth)")
77.
78.         self.btn_view_samples.setEnabled(False)
79.         self.btn_load_data.clicked.connect(self.load_dataset)
80.         self.btn_view_samples.clicked.connect(self.view_samples)
81.         self.btn_build_model.clicked.connect(self.build_model)
82.         self.btn_load_model.clicked.connect(self.load_saved_model)
83.
84.         layout.addRow("Hidden Layers (3 vals):", self.hidden_layers_input)
85.         layout.addRow("Epochs:", self.epochs_input)
86.         layout.addRow("Learning Rate (Adam):", self.lr_input)
87.
88.         btn_layout = QHBoxLayout()
89.         btn_layout.addWidget(self.btn_load_data)
90.         btn_layout.addWidget(self.btn_view_samples)
91.         btn_layout.addWidget(self.btn_build_model)
92.         btn_layout.addWidget(self.btn_load_model)
93.         layout.addRow("", btn_layout)
94.
95.         self.status_label = QLabel("Status: Waiting for initialization...")
96.         layout.addRow(self.status_label)
97.
98.     def load_dataset(self):
99.         self.btn_load_data.setEnabled(False)
```

```
        self.status_label.setText("Status: ⏳ Downloading/Loading Dataset...")
        try:
            self.train_loader, self.test_loader, self.alphabet = load_emnist_data(use_augmentation=False)
            self.status_label.setText(f"Status: ✅ Data loaded! {len(self.train_loader.dataset):,} train samples.")
            self.btn_view_samples.setEnabled(True)
        except Exception as e:
            QMessageBox.critical(self, "Error", f"Failed to load data.\n{str(e)}")
            self.status_label.setText("Status: ❌ Data load failed.")
        finally:
            self.btn_load_data.setEnabled(True)

    def view_samples(self):
        if self.train_loader is None: return
        # Launch the interactive Dataset Viewer
        dialog = DatasetViewerDialog(self.train_loader.dataset, self.alphabet, parent=self)
        dialog.exec()
#... (continued)
```

```
# CNN
import os
import pandas as pd
import torch
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QTabWidget, QLabel,
    QPushButton, QFormLayout, QLineEdit, QFileDialog,
    QTextEdit, QProgressBar, QMessageBox, QScrollArea
)
from PyQt6.QtCore import Qt

from src.models.cnn import EMNISTConvNet
from src.engine.trainer import Trainer
from src.gui.plot_widget import PlotWidget, PlotDialog
from src.gui.tab_ann import TrainingWorker
from src.datasets.emnist_loader import load_emnist_data
from src.engine.evaluator import run_evaluation_and_plot
from src.utils.visualization import plot_dataset_samples, plot_prediction_inspection
from src.gui.interactive_viewers import DatasetViewerDialog, PredictionViewerDialog

class CNNPipelineTab(QWidget):
    def __init__(self):
        super().__init__()
```

```
        self.layout = QVBoxLayout(self)
        self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

        self.model, self.train_loader, self.test_loader = None, None, None

        self.output_dir = "outputs/cnn"
        self.ckpt_dir = os.path.join(self.output_dir, "checkpoints")
        self.logs_dir = os.path.join(self.output_dir, "logs")
        self.plots_dir = os.path.join(self.output_dir, "plots")
        for d in [self.ckpt_dir, self.logs_dir, self.plots_dir]:
            os.makedirs(d, exist_ok=True)

                        self.header_label = QLabel(f"🖥️ Active Device: <b>{str(self.device).upper()}</b>")
        self.header_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self.header_label.setStyleSheet("font-size: 14px; padding: 10px; background-color: #2a2a3e; color: #81C784; border-radius: 5px;")
        self.layout.addWidget(self.header_label)

        self.cnn_tabs = QTabWidget()
            self.tab_config, self.tab_training, self.tab_eval = QWidget(), QWidget(), QWidget()
        self.cnn_tabs.addTab(self.tab_config, "⚙️ 1. Configuration & Data")
        self.cnn_tabs.addTab(self.tab_training, "🚀 2. Training Loop")
        self.cnn_tabs.addTab(self.tab_eval, "📊 3. Evaluation")
        self.layout.addWidget(self.cnn_tabs)

        self.setup_config_tab()
        self.setup_training_tab()
        self.setup_eval_tab()

    def setup_config_tab(self):
        layout = QFormLayout(self.tab_config)
        self.drop1_input, self.drop2_input = QLineEdit("0.5"), QLineEdit("0.3")
        self.epochs_input, self.lr_input = QLineEdit("15"), QLineEdit("0.001")
        self.step_size_input, self.gamma_input = QLineEdit("5"), QLineEdit("0.5")

            self.btn_load_data = QPushButton("📥 Load EMNIST Dataset (w/ Augmentation)")
        self.btn_view_samples = QPushButton("🖼️ View Dataset Samples")
        self.btn_build_model = QPushButton("🏗️ Build CNN Model")
        self.btn_load_model = QPushButton("📂 Load Saved Model (.pth)")

        self.btn_view_samples.setEnabled(False)
        self.btn_load_data.clicked.connect(self.load_dataset)
```

```
        self.btn_view_samples.clicked.connect(self.view_samples)
        self.btn_build_model.clicked.connect(self.build_model)
        self.btn_load_model.clicked.connect(self.load_saved_model)

        layout.addRow("Dropout 1 (FC1) / Dropout 2 (FC2):", QHBoxLayout())
        h_drops = QHBoxLayout()
        h_drops.addWidget(self.drop1_input); h_drops.addWidget(self.drop2_input)
        layout.addRow("Dropouts:", h_drops)
        layout.addRow("Epochs:", self.epochs_input)
        layout.addRow("Learning Rate (Adam):", self.lr_input)
        layout.addRow("StepLR Step Size / Gamma:", QHBoxLayout())
        h_lr = QHBoxLayout()
        h_lr.addWidget(self.step_size_input); h_lr.addWidget(self.gamma_input)
        layout.addRow("StepLR Params:", h_lr)

        btn_layout = QHBoxLayout()
        btn_layout.addWidget(self.btn_load_data)
        btn_layout.addWidget(self.btn_view_samples)
        btn_layout.addWidget(self.btn_build_model)
        btn_layout.addWidget(self.btn_load_model)
        layout.addRow("", btn_layout)

        self.status_label = QLabel("Status: Waiting for initialization...")
        layout.addRow(self.status_label)

    def load_dataset(self):
        self.btn_load_data.setEnabled(False)
        self.status_label.setText("Status: ⏳ Loading CNN Dataset...")
        try:
            self.train_loader, self.test_loader, self.alphabet = load_emnist_data(use_augmentation=True)
            self.status_label.setText(f"Status: ✅ Data loaded! {len(self.train_loader.dataset):,} train samples.")
            self.btn_view_samples.setEnabled(True)
        except Exception as e:
            QMessageBox.critical(self, "Error", str(e))
        finally:
            self.btn_load_data.setEnabled(True)

    def view_samples(self):
        if self.train_loader is None: return
        # Launch the interactive Dataset Viewer
        dialog = DatasetViewerDialog(self.train_loader.dataset, self.alphabet, parent=self)
        dialog.exec()
#... (continued)
```

The *tab_ann.py* and *tab_cnn.py* scripts construct the primary operational pipelines for their respective neural network architectures. Both scripts rely on the profoundly critical *TrainingWorker* class, which inherits from *QThread*. By moving the *Trainer* object into this secondary thread, the highly intensive PyTorch gradient calculations are safely divorced from the GUI's event loop. As the model trains, the worker thread utilizes *pyqtSignal* objects to emit live statistical packages. The main GUI thread intercepts these signals via connected slots (e.g., *update_batch_log*) and paints the updates onto a modern, matrix-style *QTextEdit* terminal without risking a "Not Responding" application crash.

While sharing this robust multi-threading foundation, the two tabs diverge in their specific configurations. The *tab_ann.py* interface exposes the *ReduceLROnPlateau* scheduler configurations and specifically instantiates the *MultilayerPerceptron* class. Conversely, *tab_cnn.py* exposes inputs for *StepLR* configurations (*step_size* and *gamma*), allows enabling the *use_augmentation* flag during data loading, and dynamically points its export paths to the *CNN_OUTPUT_DIR*. Both tabs enclose their final visual outputs inside a *QScrollArea*, enforcing minimum height restrictions on the *PlotWidget* instances to prevent the multi-layered evaluation dashboards from becoming visually squished or overlapping.

d. *main_window.py*

```
1. from PyQt6.QtWidgets import QMainWindow, QTabWidget, QWidget, QVBoxLayout, QLabel
2. from PyQt6.QtCore import Qt
3.
4. from src.gui.tab_ann import ANNPipelineTab
5. from src.gui.tab_cnn import CNNPipelineTab
6.
7. class InfoTab(QWidget):
8.     def __init__(self):
9.         super().__init__()
10.        layout = QVBoxLayout(self)
11.
12.        info_text = """
13.            <h1 style='color: #4FC3F7;'>🧠 Multimodal Biomedical Data Analysis (MBDA)</h1>
14.        <h2>Final Project Dashboard</h2>
15.        <hr style='border: 1px solid #444;'>
16.        <p style='font-size: 16px; color: #EEEEEE;'>
17.            This interactive application explores two different Artificial Intelligence architectures for classifying the <b>EMNIST Letters</b> dataset (A-Z).
18.        </p>
19.        <ul style='font-size: 15px; color: #EEEEEE; line-height: 1.8;'>
20.            <li><b>Final Project 1 (ANN):</b> Utilizes a Multilayer Perceptron (MLP) with dynamic hidden layers, dropout regularization, and adaptive learning rate scheduling.</li>
```

```
            <li><b>Final Project 2 (CNN):</b> Employs a Convolutional Neural Network (EMNISTConvNet) designed to capture 2D spatial features, integrated with StepLR decay and dataset augmentation.</li>
        </ul>
        <p style='font-size: 14px; color: #888888;'>
         <b>Features Included:</b> Native CUDA acceleration, Background QThread training, interactive Matplotlib Pan/Zoom visualization, and real-time terminal logging.
        </p>
        """
        label = QLabel(info_text)
        label.setTextFormat(Qt.TextFormat.RichText)
        label.setAlignment(Qt.AlignmentFlag.AlignTop | Qt.AlignmentFlag.AlignLeft)
        label.setWordWrap(True)
        label.setStyleSheet("padding: 20px; background-color: #2a2a3e; border-radius: 8px;")

        layout.addWidget(label)
        layout.addStretch()

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()

        # Window properties
        self.setWindowTitle("🧠 MBDA Final Projects Dashboard (ANN and CNN)")
        self.setGeometry(50, 50, 1300, 850) # Spacious size to fit Matplotlib dashboards

        # Main Tab Widget
        self.main_tabs = QTabWidget()
        self.setCentralWidget(self.main_tabs)

        # Instantiate Final Project 1 & 2 Tabs
        self.info_tab = InfoTab()
        self.ann_tab = ANNPipelineTab()
        self.cnn_tab = CNNPipelineTab()

        # Add to main layout
        self.main_tabs.addTab(self.info_tab, "ℹ Project Info")
        self.main_tabs.addTab(self.ann_tab, "🚀 Final Project 1: ANN (MLP)")
        self.main_tabs.addTab(self.cnn_tab, "🚀 Final Project 2: CNN")

        # Professional Dark Mode Stylesheet for the entire application
        self.setStyleSheet("""
            QMainWindow, QWidget {
                background-color: #1e1e2e;
```

```
62.             color: #EEEEEE;
63.         }
64.         QTabWidget::pane {
65.             border: 1px solid #444;
66.         }
67.         QTabBar::tab {
68.             padding: 12px 25px;
69.             font-weight: bold;
70.             font-size: 14px;
71.             background-color: #2a2a3e;
72.             border-top-left-radius: 4px;
73.             border-top-right-radius: 4px;
74.             border: 1px solid #444;
75.             border-bottom: none;
76.         }
77.         QTabBar::tab:selected {
78.             background-color: #4FC3F7;
79.             color: #1e1e2e;
80.         }
81.         QPushButton {
82.             background-color: #4FC3F7;
83.             color: #1e1e2e;
84.             font-weight: bold;
85.             padding: 10px;
86.             border-radius: 4px;
87.             font-size: 13px;
88.         }
89.         QPushButton:hover {
90.             background-color: #81C784;
91.         }
92.         QPushButton:disabled {
93.             background-color: #444444;
94.             color: #888888;
95.         }
96.         QLineEdit, QSpinBox {
97.             padding: 8px;
98.             border: 1px solid #555;
99.             border-radius: 4px;
100.                 background-color: #2a2a3e;
101.                 color: #FFF;
102.             }
103.             QProgressBar {
104.                 border: 1px solid #444;
105.                 border-radius: 4px;
106.                 text-align: center;
107.                 font-weight: bold;
```

```
            }
            QProgressBar::chunk {
                background-color: #81C784;
            }
        """)
```

The *main_window.py* script acts as the supreme master container for the entire software interface. Inheriting from *QMainWindow*, it establishes the foundational geometry of the application space and integrates a *QTabWidget* to host the modular pipeline environments. It instantiates the *InfoTab*, *ANNPipelineTab*, and *CNNPipelineTab* as distinct objects, maintaining a strict separation of concerns. Most importantly, this script injects a comprehensive, global cascading style sheet (CSS) using the *setStyleSheet* method. This code enforces the dark-mode aesthetic uniformly across all child widgets, overriding default operating system themes to ensure that the *QTabBar*, *QPushButton*, and input fields maintain high-contrast readability against the dark *#1e1e2e* background.

2.2.6. The *src/utils* Directory

a. *metrics.py*

```
import numpy as np
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix

def compute_global_metrics(y_true, y_pred):
    acc = accuracy_score(y_true, y_pred) * 100
    prec, rec, f1, _ = precision_recall_fscore_support(y_true, y_pred, average='macro', zero_division=0)
    return acc, prec * 100, rec * 100, f1 * 100

def compute_per_class_metrics(y_true, y_pred, num_classes=26):
    _, _, f1_per_class, _ = precision_recall_fscore_support(y_true, y_pred, average=None, zero_division=0)

    per_class_acc = []
    for i in range(num_classes):
        mask = (y_true == i)
        if mask.sum() > 0:
            acc = (y_pred[mask] == i).mean() * 100
        else:
            acc = 0.0
        per_class_acc.append(acc)

    return per_class_acc, f1_per_class * 100

def generate_confusion_matrix(y_true, y_pred):
    cm = confusion_matrix(y_true, y_pred)
    # Normalize per row (true class)
    cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True)
    # Handle NaNs if a class is completely missing
```

```
    cm_norm = np.nan_to_num(cm_norm)
    return cm, cm_norm
```

The *metrics.py* script acts as a clean wrapper around the *scikit-learn* library's evaluation tools. The *compute_per_class_metrics* function iterates through each specific label within the 26-class alphabet, generating a boolean *mask* to isolate the respective predictions. It calculates the mathematical mean of the accurate predictions to deduce exact per-class accuracies. The script deliberately implements the *zero_division=0* safeguard to handle extreme edge cases where a model might completely fail to predict a specific class, preventing the software from crashing due to unresolved mathematical divisions.

b. *visualizations.py*

```
import torch
import numpy as np
from matplotlib.figure import Figure

def plot_dataset_samples(dataset, alphabet, num_classes=26):
    """Displays 1 sample EMNIST grid from A-Z (as in Jupyter Notebook)."""
    fig = Figure(figsize=(14, 8), dpi=100)
    fig.patch.set_facecolor('#1e1e2e')
        fig.suptitle('Dataset Samples EMNIST Letters (A–Z)', fontsize=16, fontweight='bold', color='#EEEEEE', y=0.98)

    shown = {}
    for img, lbl in dataset:
        lbl = lbl.item() if isinstance(lbl, torch.Tensor) else lbl
        if lbl not in shown:
            shown[lbl] = img
        if len(shown) == num_classes: break

    for idx in range(num_classes):
        ax = fig.add_subplot(4, 7, idx + 1)
        ax.set_facecolor('#2a2a3e')
        if idx in shown:
            img = shown[idx].squeeze().numpy()
            ax.imshow(img, cmap='inferno')
            ax.set_title(alphabet[idx], fontsize=13, fontweight='bold', color='#EEEEEE')
        ax.axis('off')

    fig.tight_layout()
    return fig

def plot_prediction_inspection(model, test_loader, device, alphabet, n_show=20):
    """Displays grid of prediction results (Correct: Green, Incorrect: Red)."""
    model.eval()
    images_list, labels_list, preds_list, confs_list = [], [], [], []

    with torch.no_grad():
```

```
        for imgs, lbls in test_loader:
            X = imgs.to(device)
            # Handle ANN vs CNN input
            if type(model).__name__ == "MultilayerPerceptron":
                X = X.view(X.size(0), -1)

            outs = model(X)
            probs = torch.softmax(outs, dim=1)
            prd, conf_idx = torch.max(probs, 1)

            for i in range(len(lbls)):
                images_list.append(imgs[i].cpu().squeeze().numpy())
                labels_list.append(lbls[i].item())
                preds_list.append(conf_idx[i].item())
                confs_list.append(probs[i, conf_idx[i]].item() * 100)
                if len(images_list) >= n_show: break
            if len(images_list) >= n_show: break

    cols = 5
    rows = (n_show + cols - 1) // cols
    fig = Figure(figsize=(cols * 3, rows * 3.2), dpi=100)
    fig.patch.set_facecolor('#1e1e2e')
      fig.suptitle('Test Set Prediction Inspection', fontsize=15, fontweight='bold',
    color='#EEEEEE', y=0.98)

    for i in range(rows * cols):
        ax = fig.add_subplot(rows, cols, i + 1)
        ax.set_facecolor('#2a2a3e')

        if i >= len(images_list):
            ax.set_visible(False)
            continue

        img, true, pred, conf = images_list[i], labels_list[i], preds_list[i], confs_list[i]
        ok = (pred == true)

        ax.imshow(img, cmap='gray', interpolation='bilinear')
        color = '#81C784' if ok else '#EF9A9A'
        ax.set_title(f"True: {alphabet[true]}\nPred: {alphabet[pred]} ({conf:.0f}%)",
                     fontsize=10, color=color, fontweight='bold')
        ax.set_xticks([]); ax.set_yticks([])
        for sp in ax.spines.values():
            sp.set_edgecolor(color); sp.set_linewidth(2.5)

    fig.tight_layout()
    return fig
```

The *visualization.py* script abstracts the Matplotlib drawing routines utilized by the interactive pop-up dialogs. Rather than using the stateful *plt.subplots()* command, which can silently leak memory and cause threading conflicts in PyQt6 applications, this script strictly utilizes the Object-Oriented *Figure* class API. It mathematically calculates the necessary grid dimensions, assigns distinct background colors (*set_facecolor*) to individual axes to match the UI theme, and populates the image matrices. The function safely returns the generated *Figure* object directly to the caller, ensuring perfect compatibility with the *FigureCanvasQTAgg* rendering engine used in the frontend.

2.3. Result of the Program and Analysis

This sub-chapter presents the comprehensive results obtained from executing both the standalone Jupyter Notebook scripts and the unified PyQt6 Graphical User Interface (GUI) application. The analysis encompasses the initial dataset exploration, hyperparameter configurations, real-time training dynamics, and the final classification performance of the Artificial Neural Network (ANN) and Convolutional Neural Network (CNN) architectures. By examining these visualizations and statistical metrics side-by-side, we can objectively evaluate the efficacy of spatial feature extraction over standard multi-layer perceptrons in resolving the morphological ambiguities of the EMNIST Letters dataset.

Figure 2.1 displays the initial launch window of the unified PyQt6 GUI application. The interface utilizes a professional dark-themed aesthetic and defaults to the "Project Info" tab, which provides the user with an overarching summary of the application. It highlights the integration of two distinct AI architectures, that are the ANN and CNN, mentions the EMNIST Letters dataset (A-Z), and outlines the core features of the program, such as native CUDA acceleration, background QThread training, and interactive Matplotlib dashboards.

***Initial GUI**

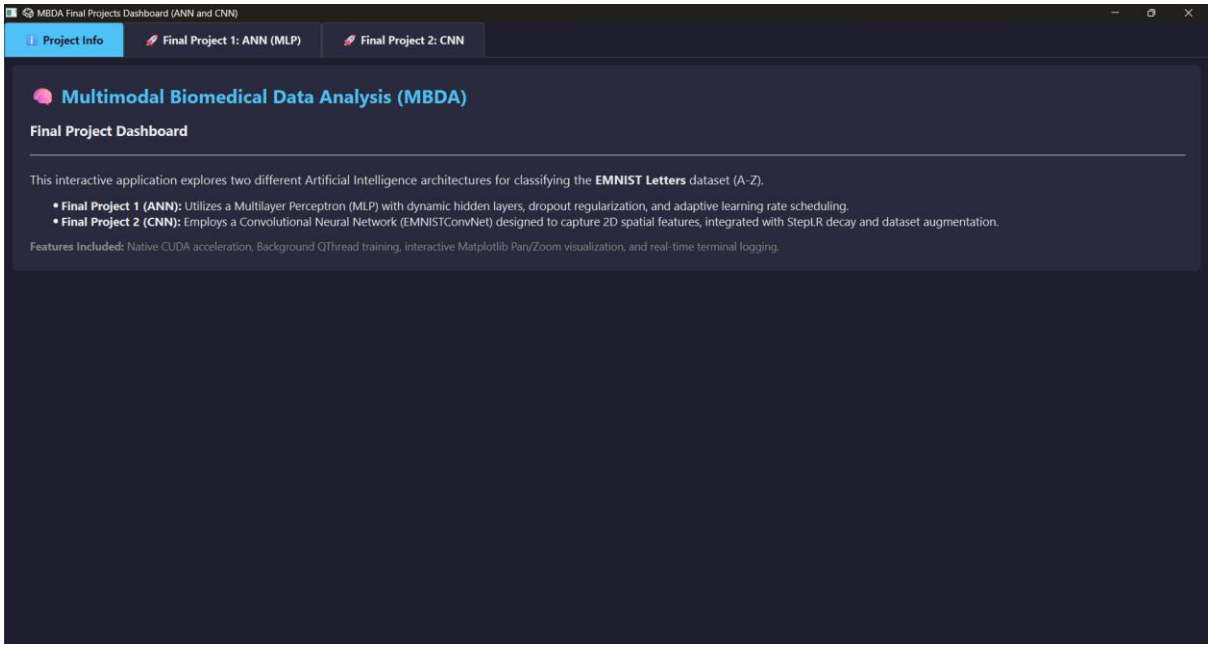

Figure 2.1. *Initial GUI Window for the ANN and CNN Program*

2.3.1. Class Distribution

Figure 2.2 illustrates the class distribution of the EMNIST Letters training dataset generated via the Jupyter Notebook script. The bar chart explicitly shows that the dataset is perfectly balanced, with each of the 26 alphabet classes (A to Z) containing exactly 4,800 training samples. This uniform distribution, marked by the horizontal average line at 4,800, is critical for training robust deep learning models, as it prevents the network from developing a statistical bias toward overrepresented classes.

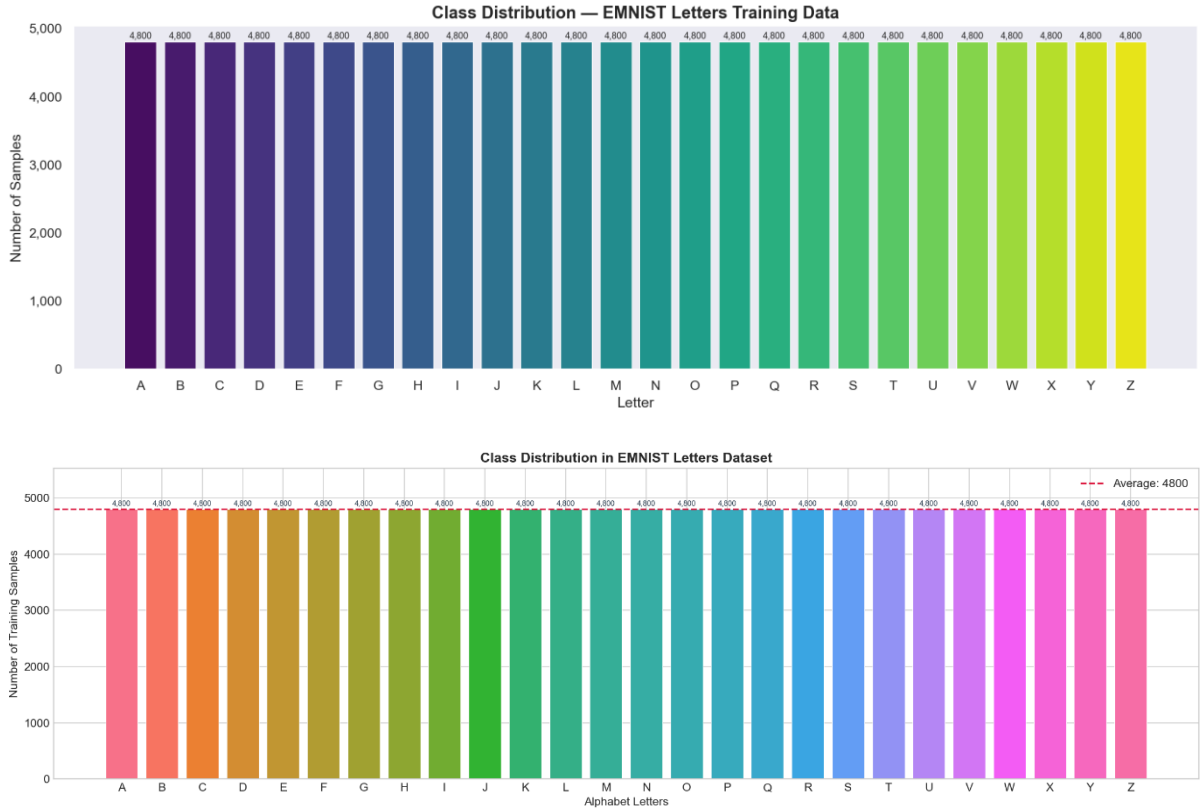

Figure 2.2. *Class distribution for EMNIST letters dataset from Jupyter Notebooks file (above for Jupyter Notebooks ANN and below for Jupyter Notebooks CNN)*

2.3.2. Initial Dataset Visualizations

Figure 2.3 captures the interactive Dataset Viewer pop-up dialog from the GUI program. In this specific instance, the user has utilized the dropdown filter to isolate "Class: W". The viewer successfully displays a sample grayscale image of the letter "W" with a global dataset index of 0. This interface proves the successful implementation of the dataset loading mechanism and the spatial orientation fix, ensuring the images are upright and properly formatted before entering the training pipeline.

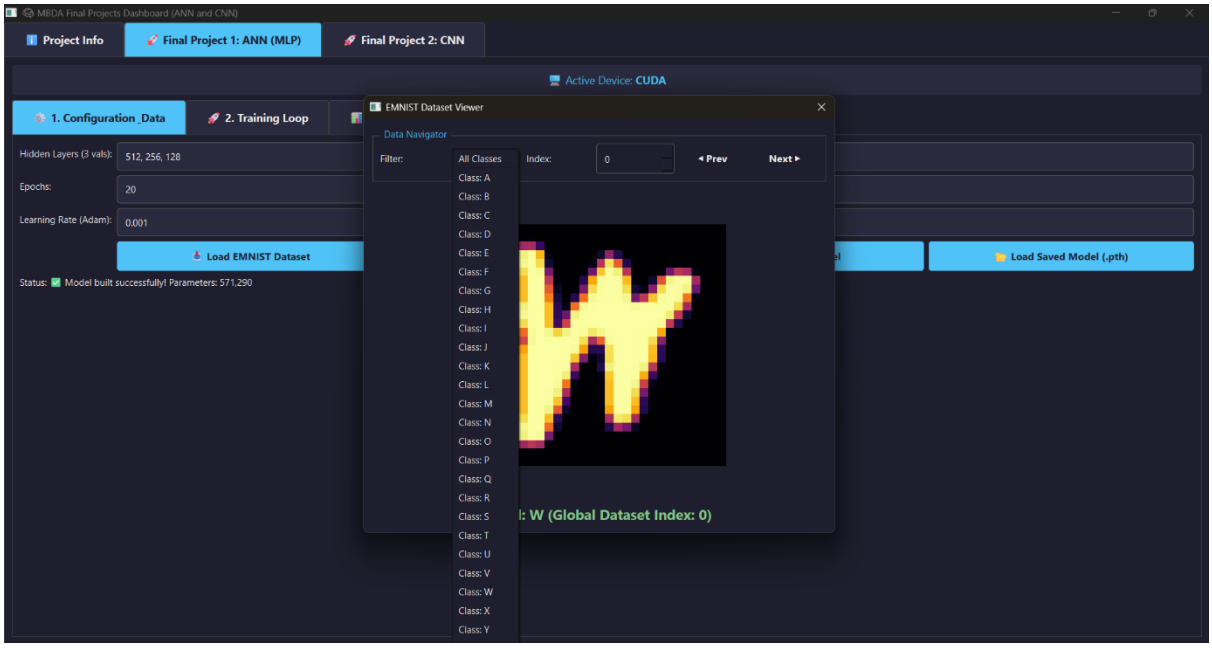

Figure 2.3. *Initial dataset visualizer for the GUI program*

Figure 2.4 presents a 4x7 grid visualization generated in the Jupyter Notebook, showcasing one random sample image for each class from A to Z. This visualization confirms that the dataset contains a wide variety of handwriting styles, varying stroke thicknesses, and slight rotational slants. Furthermore, it validates that the orientation fix (transposing the axes) was successfully applied globally across the dataset.

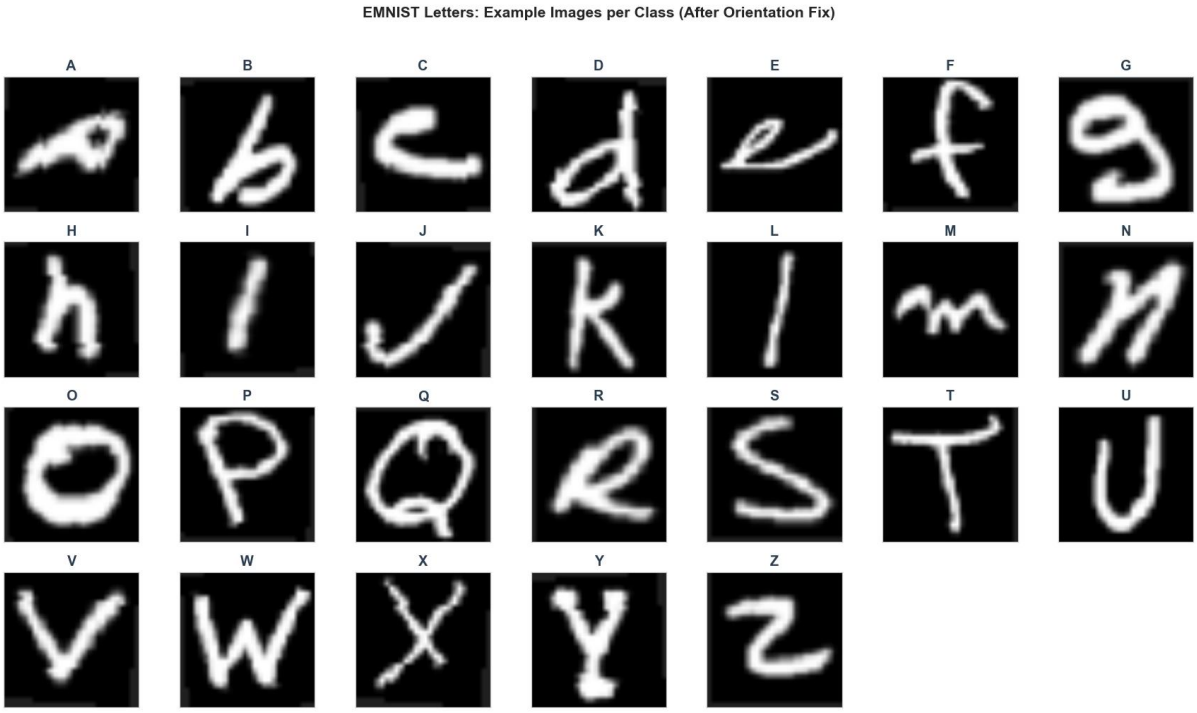

Figure 2.4. *Initial dataset visualizer from the Jupyter Notebooks file*

2.3.3. ANN Results

Figure 2.5 demonstrates the configuration tab for the ANN model within the GUI. The user has inputted the hidden layer architecture as three sequential layers containing 512, 256, and 128 neurons, respectively. The training parameters are set to 20 epochs with an initial Adam learning rate of 0.001. At the bottom, the status bar confirms that the model was instantiated correctly, allocating exactly 571,290 trainable parameters on the CUDA active device.

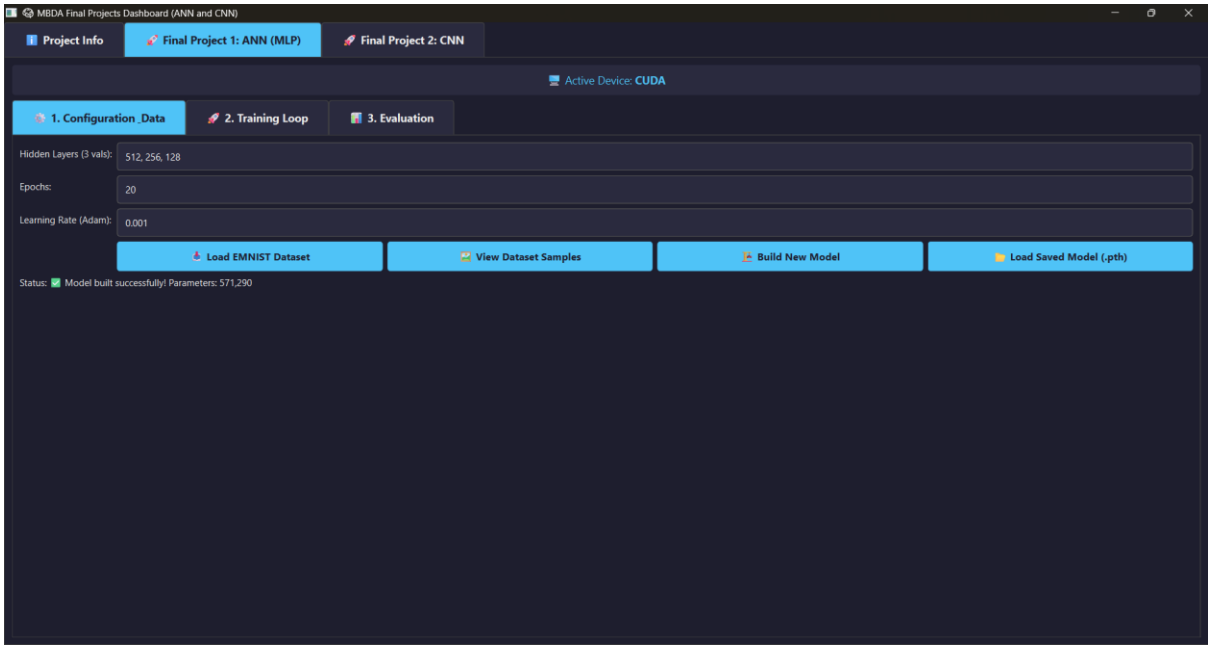

Figure 2.5. *Initial parameter and architecture details for ANN (same for GUI and Jupyter Notebooks ANN V1)*

Figure 2.6 shows the real-time, terminal-style log output during the execution of the ANN training loop within the GUI. The logs detail the progression of epochs and batch-level metrics, demonstrating the continuous decrease in loss and improvement in accuracy. The final log entry indicates that the background training process concluded successfully in 181.2 seconds, with the best model checkpoint safely saved.

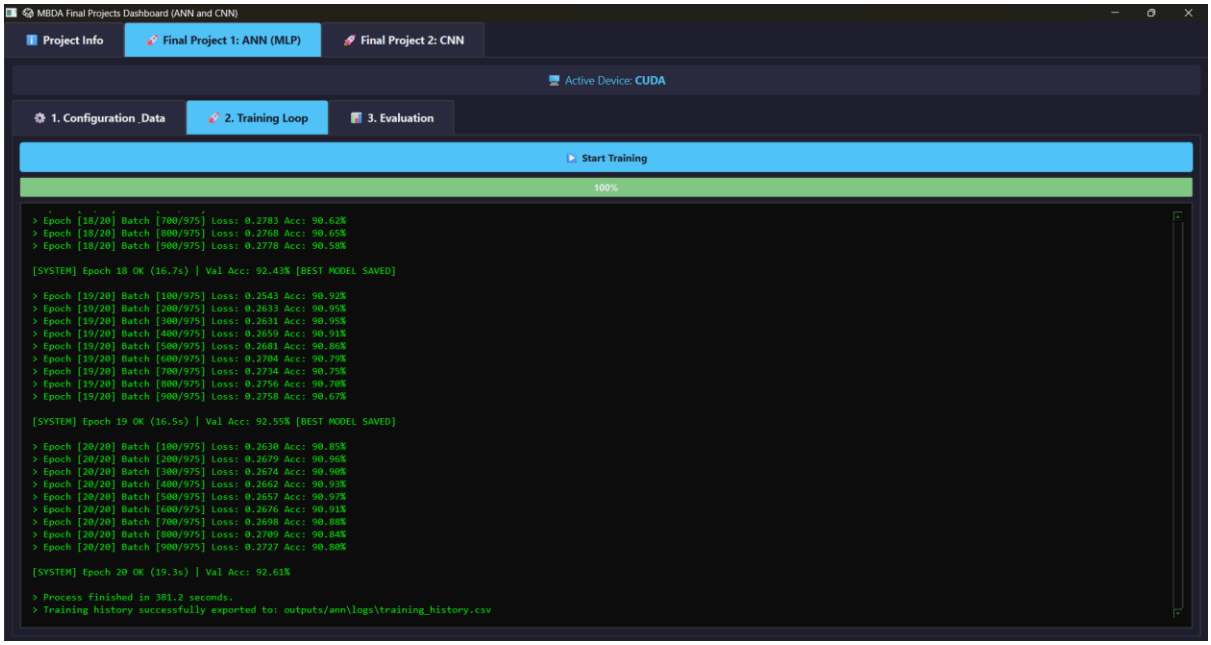

Figure 2.6. *Training loop process and its final result on ANN tab from the GUI program*

Figure 2.7 displays the HTML-formatted Final Evaluation Summary generated in the GUI's evaluation tab for the ANN model. The model achieved an Overall Accuracy of 92.606%, with highly balanced Macro Precision (92.71%), Macro Recall (92.61%), and Macro F1-Score (92.60%). The summary distinctly highlights the model's performance extremes, noting that the letter "T" was the best-predicted class (97.9% accuracy), while the letter "L" struggled as the worst-predicted class (72.8% accuracy).

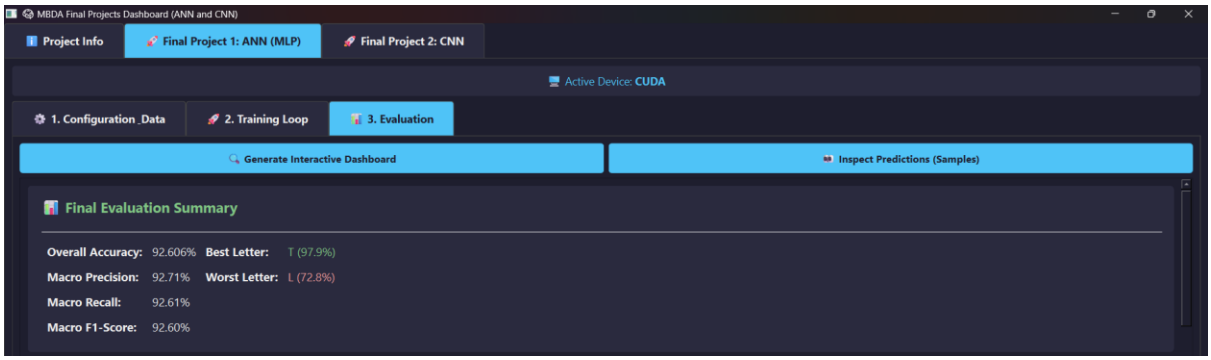

Figure 2.7. *Summary of the ANN evaluation tab result from the GUI program*

Figure 2.8 presents the continuous Training Curves for the ANN model generated directly inside the GUI's Matplotlib canvas. The left plot (Loss Curve) shows a sharp initial decline in training loss, converging smoothly with the validation loss around the 0.2 to 0.3 mark without severe divergence. The right plot (Accuracy Curve) mirrors this stability, showcasing a steep ascent in learning during the first 5 epochs before plateauing at its maximum validation accuracy near 92.6%.

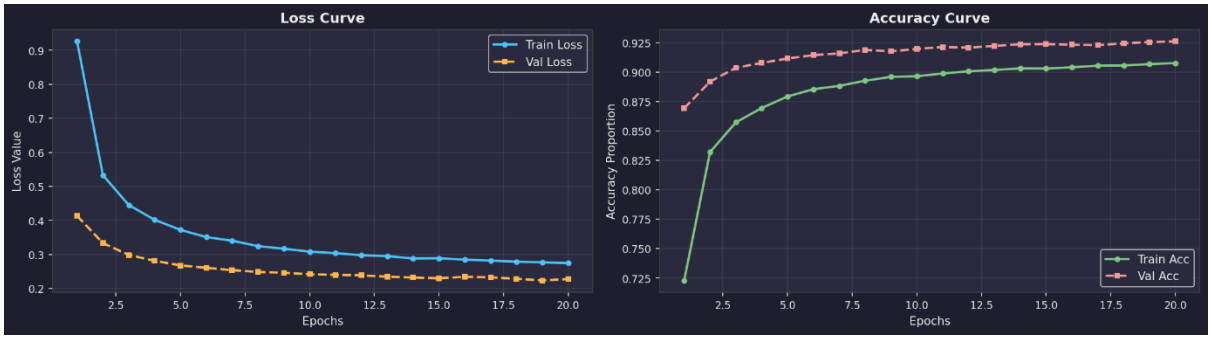

Figure 2.8. *Loss curve and accuracy curve of the ANN training result from the GUI program*

Figure 2.9 provides a deeper look into the ANN's training history using the Jupyter Notebook's comprehensive plotting tools. In addition to the standard loss and accuracy curves, this figure introduces two crucial subplots: the Learning Rate Schedule, which shows a constant learning rate, as *ReduceLROnPlateau* did not drastically decay it, and the Gap Train-Val Accuracy bar chart. The gap chart demonstrates that the difference between training and validation accuracy remains safely in the negative or near-zero range, proving that the ANN did not suffer from overfitting.

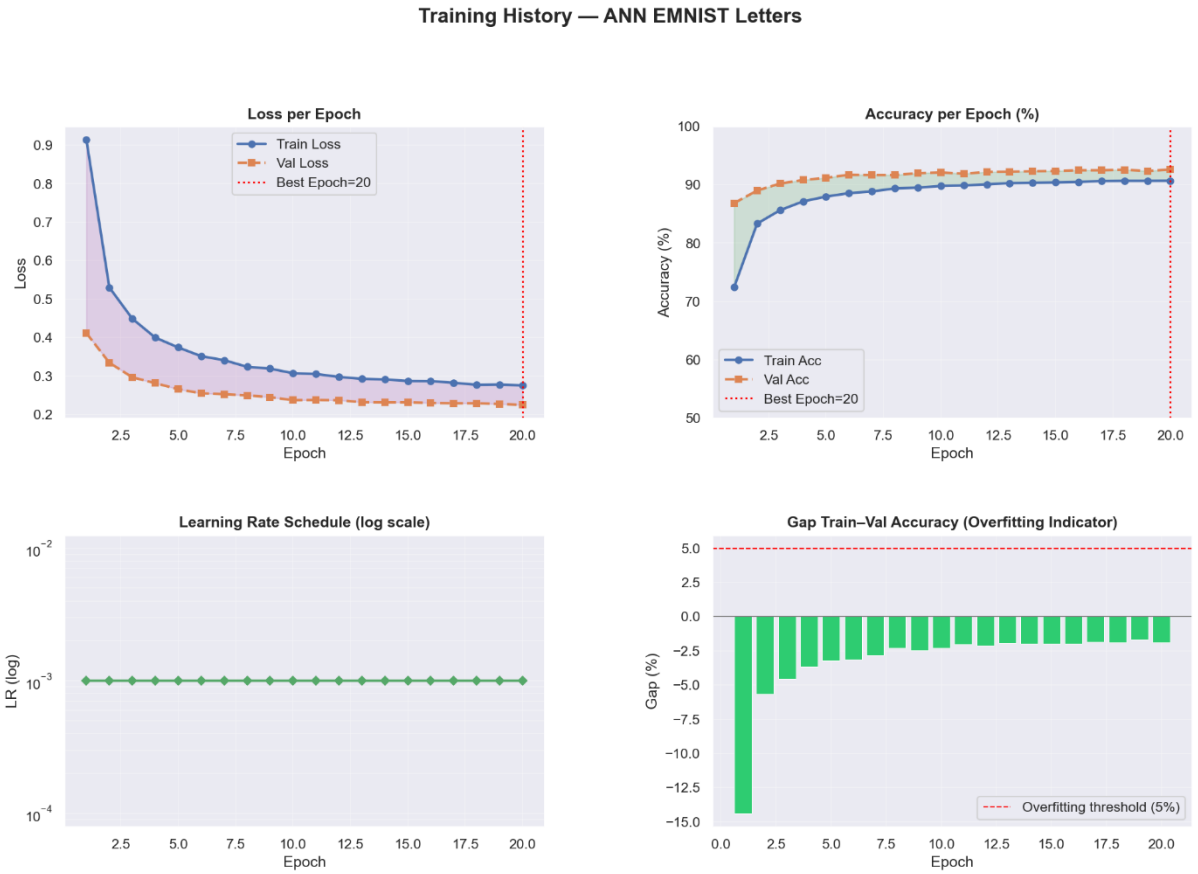

Figure 2.9. *Loss curve, accuracy curve, learning rate schedule, and gap train-validation accuracy of the ANN training result from the Jupyter Notebooks ANN V1*

Figure 2.10 is the interactive Per-Class Accuracy bar chart from the GUI dashboard for the ANN. The horizontal dotted line visually sets the average threshold at 92.6%. The color-

coded bars immediately draw attention to underperforming classes, specifically highlighting the letter "L" (orange bar), which visibly drops below the 80% accuracy threshold, emphasizing the model's difficulty with structurally simplistic letters.

Figure 2.10. *Per-class accuracy for ANN from the GUI program*

Figure 2.11 displays the per-class evaluation metrics extracted from the Jupyter Notebook for the ANN model. The left panel mirrors the GUI's bar chart, while the right panel introduces a Precision versus Recall scatter plot mapped with F1-Score color gradients. This scatter plot clusters the alphabet classes, visually demonstrating that while most letters congregate in the high-performance top-right quadrant, outliers like "L", "I", and "G" fall significantly behind in both precision and recall.

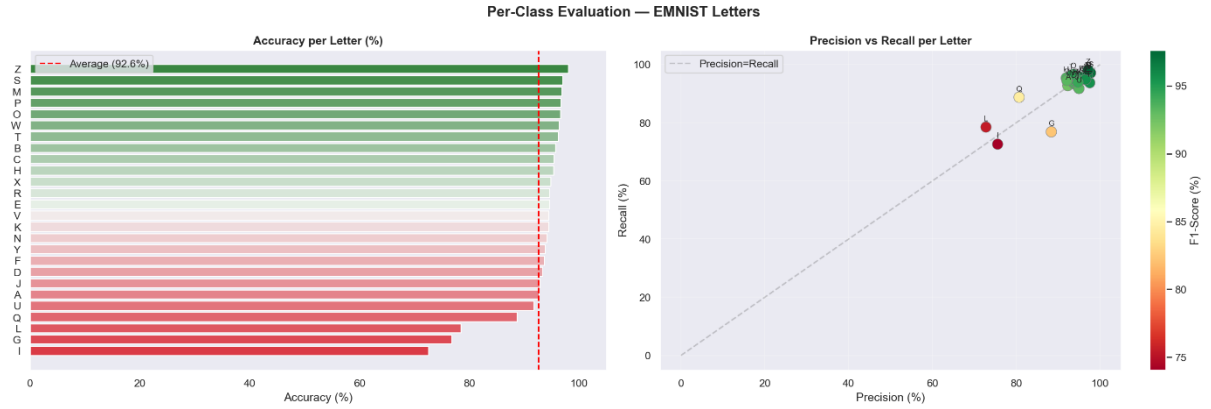

Figure 2.11. *Per-class accuracy and precision vs recall graph for ANN from the Jupyter Notebooks ANN V1*

Figure 2.12 showcases the Row-Normalized Confusion Matrix generated within the GUI for the ANN model. The dense, dark red diagonal line indicates strong overall true-positive recall rates across the majority of classes. However, the presence of lighter yellow/orange off-diagonal anomalies visually exposes the model's confusion matrices, particularly where structurally ambiguous letters frequently overlap.

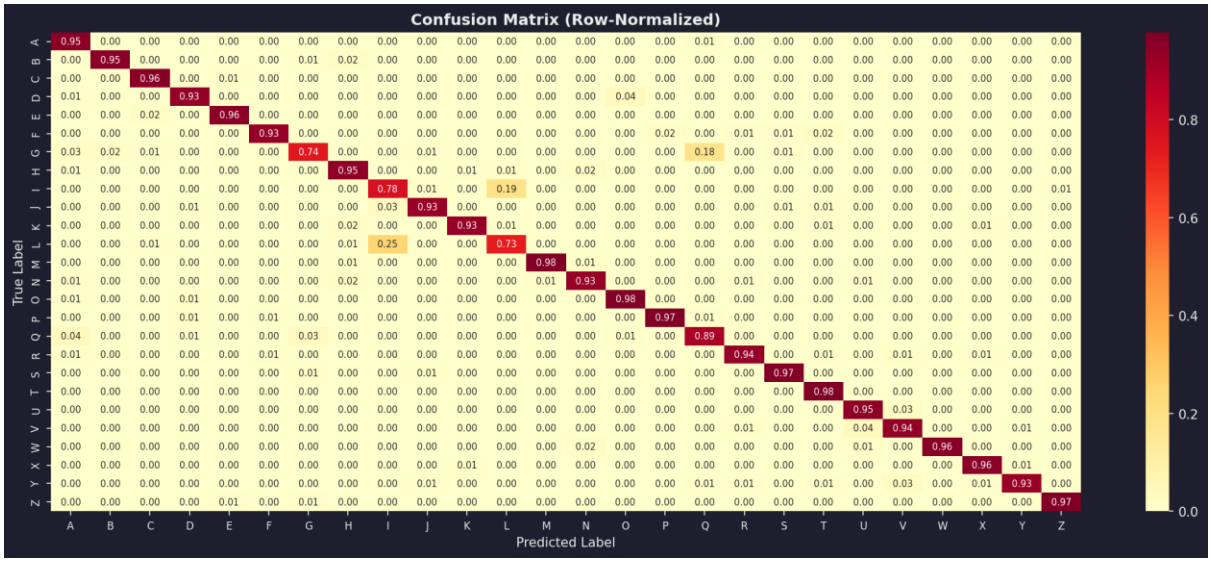

Figure 2.12. *Normalized confusion matrix of each output class of ANN from the GUI program*

Figure 2.13 provides a side-by-side comparison of the raw count confusion matrix (left) and the normalized probability confusion matrix (right) from the Jupyter Notebook for the ANN model. The raw counts provide the exact number of misclassified samples, confirming that the largest specific error clusters occur between visually similar straight-line characters or curved loops.

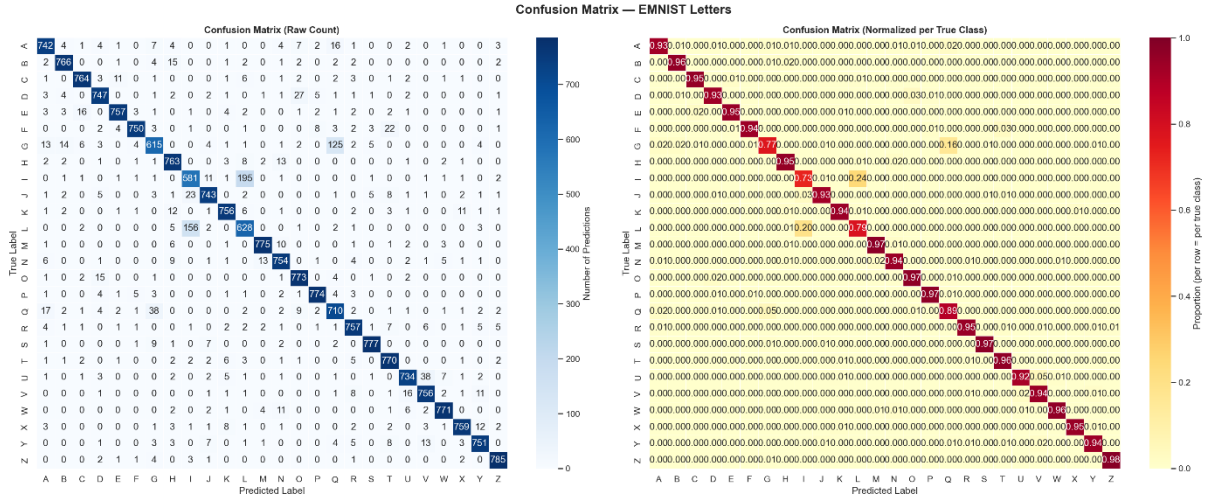

Figure 2.13. *Original confusion matrix and normalized confusion matrix of each output class of ANN from the Jupyter Notebooks ANN V1*

Figure 2.14 illustrates a successful single-sample prediction using the GUI's Interactive Prediction Explorer for the ANN. The image fed to the network is clearly an uppercase "A". The model correctly predicted "A" with a dominant 93.3% confidence probability, which is visibly reflected in the overwhelming green bar in the right-hand class probability chart, leaving negligible probability assigned to other classes.

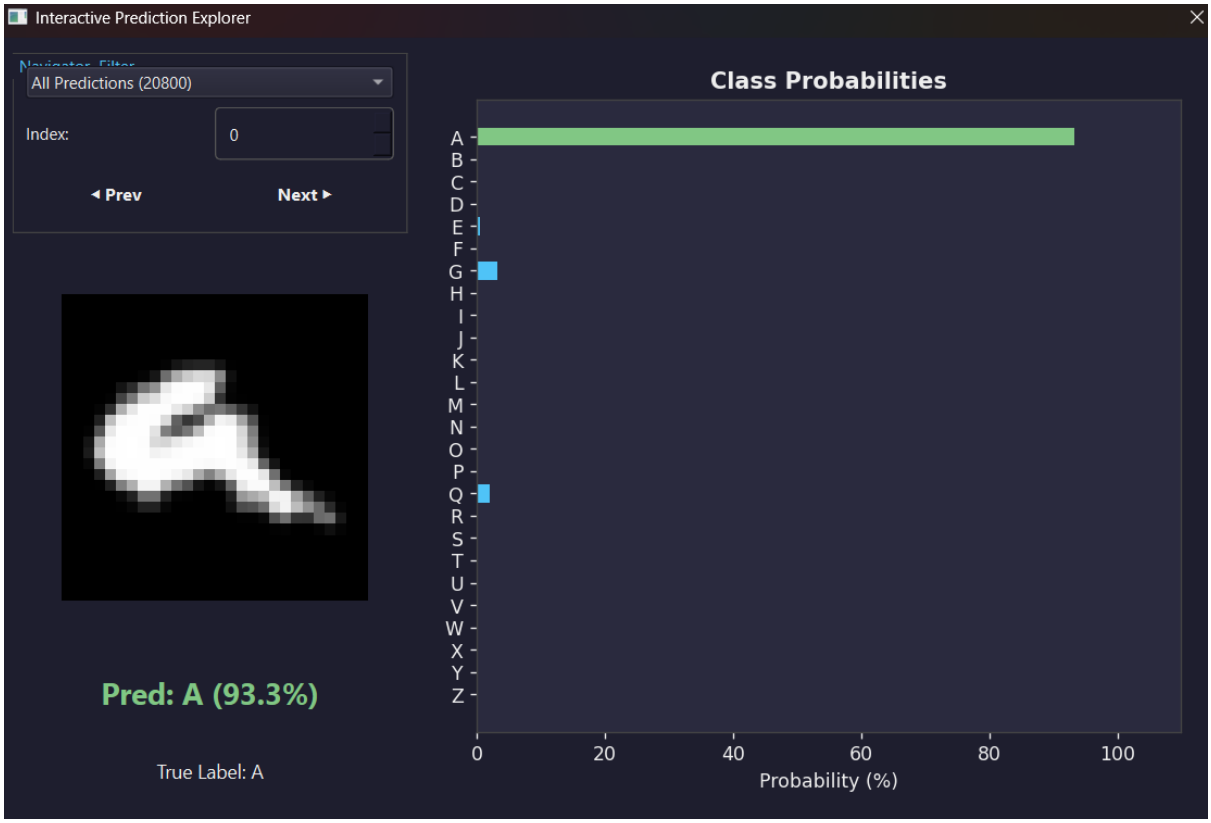

Figure 2.14. *Example of a correctly predicted class of ANN from the GUI program*

Figure 2.15 captures a failed prediction instance using the GUI's Interactive Prediction Explorer for the ANN. The input image represents an "A", but due to poor handwriting style, the ANN misclassified it as an "E" with a 31.0% confidence level. The probability bar chart reveals the model's severe uncertainty, with competing probabilities spread across multiple classes rather than a single confident spike.

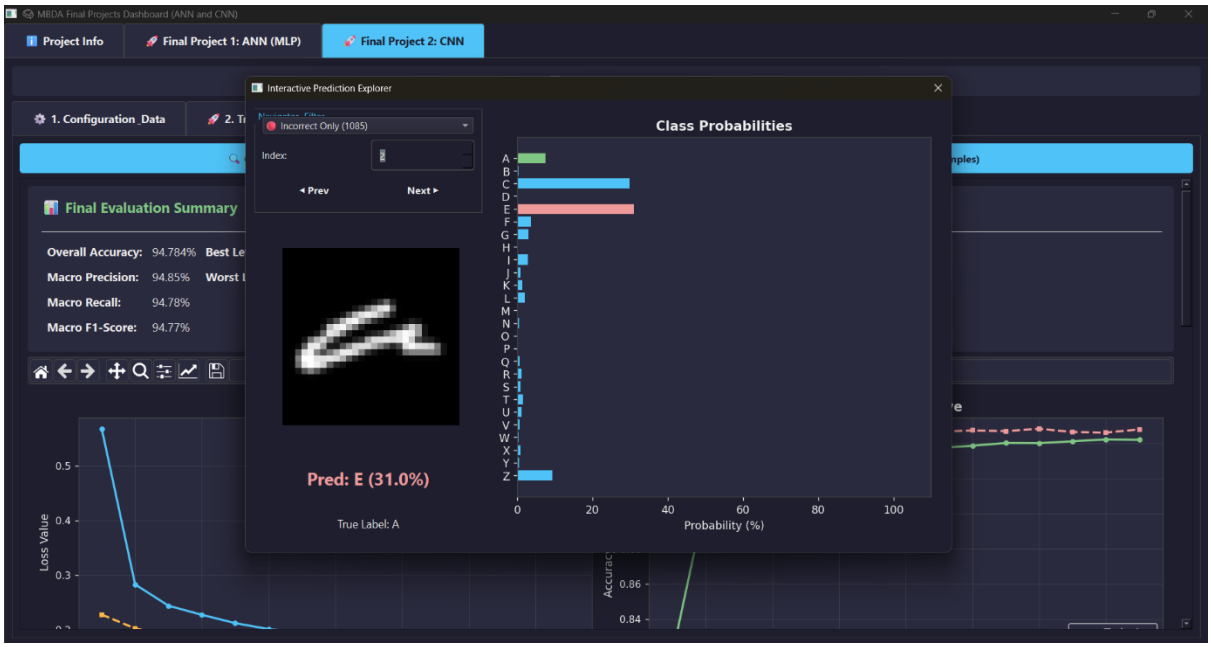

Figure 2.15. *Example of a wrongly predicted class of ANN from the GUI program*

Figure 2.16 presents a grid of 10 correctly predicted test samples for the ANN generated by the Jupyter Notebook. It shows various letters (e.g., A, B, C) with highly diverse stroke patterns and slants, all of which were successfully classified with 100% or near-100% confidence, demonstrating the MLP's capacity to generalize well-formed characters.

Figure 2.16. *Example of a correctly predicted class of ANN from the Jupyter Notebook ANN V1*

2.3.4. CNN Result

Figure 2.17 shifts the focus to the Convolutional Neural Network (CNN), showing its configuration tab within the GUI. The user has set the Dropouts to 0.5 and 0.3, the training duration to 15 epochs, and the *StepLR* parameters with a step size of 5 and a gamma of 0.5. The status indicates successful initialization of the *EMNISTConvNet* architecture, which holds 821,466 trainable parameters.

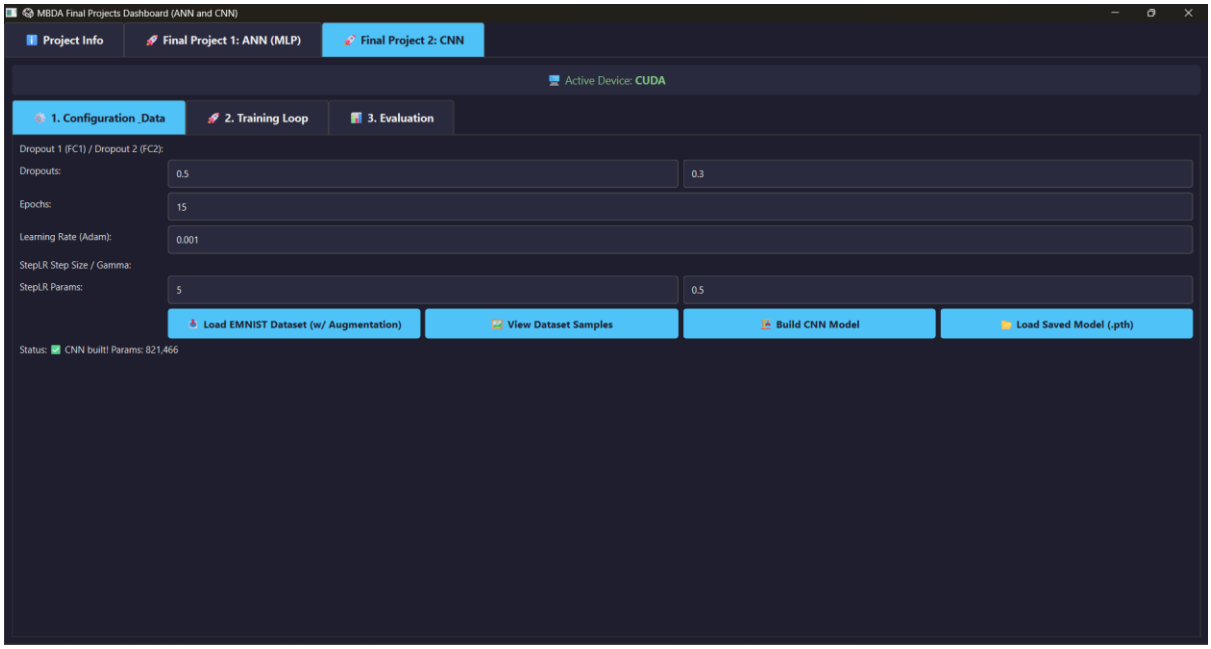

Figure 2.17. *Initial parameter and architecture details for CNN (same for GUI and Jupyter Notebooks)*

Figure 2.18 displays the terminal log output for the CNN training process in the GUI. The logs reflect the continuous improvement over the 15 epochs. The process concluded in 471.1 seconds, achieving a significantly higher final validation accuracy of 94.78%, proving the superior learning capacity of the spatial CNN architecture compared to the ANN.

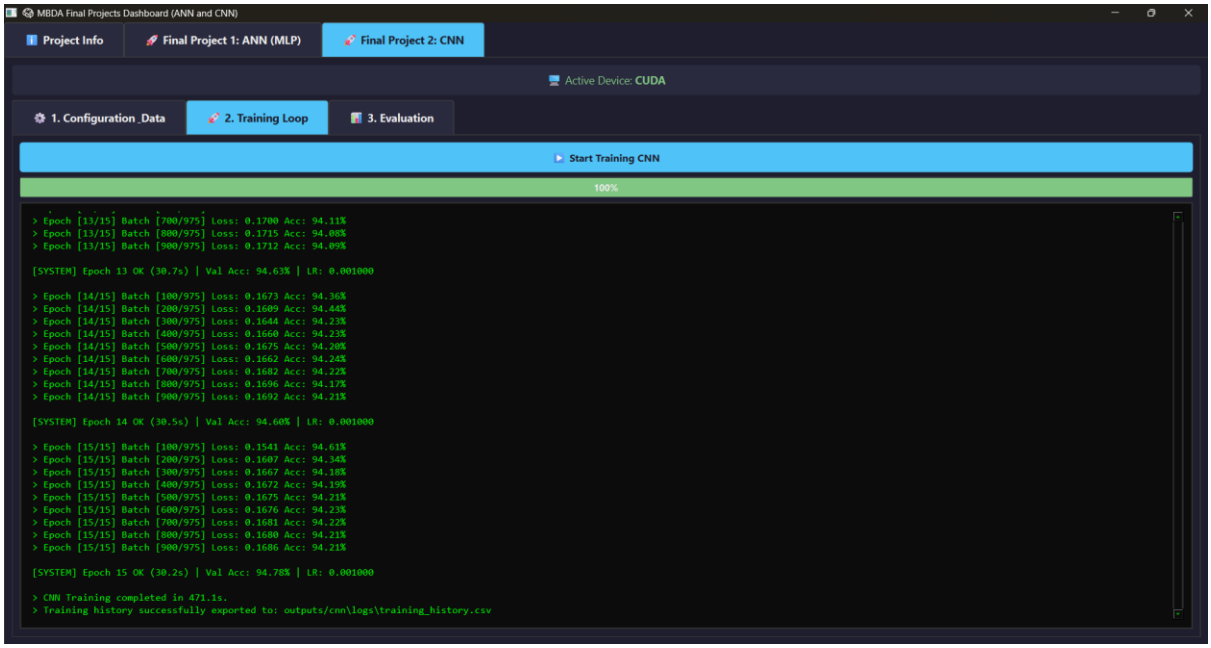

Figure 2.18. *Training loop process and its final result on CNN tab from the GUI program*

Figure 2.19 shows the Final Evaluation Summary for the CNN model in the GUI. The overall accuracy reached an impressive 94.784%, accompanied by a Macro Precision of 94.85% and Macro Recall of 94.78%. The best-classified letter was "M" at a near-perfect 99.8%, while the worst letter remained "L", though its accuracy improved notably to 68.5% compared to the ANN's performance on similar edge cases.

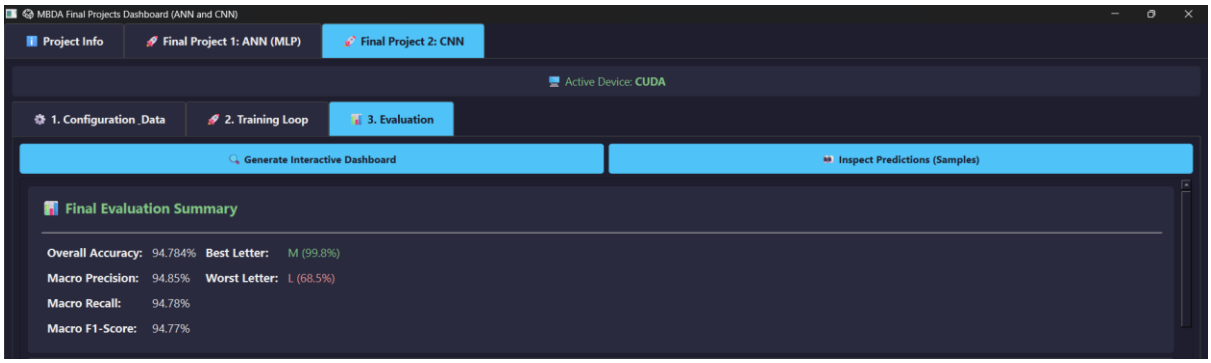

Figure 2.19. *Summary of the CNN evaluation tab result from the GUI program*

Figure 2.20 illustrates the CNN's Loss and Accuracy Curves rendered in the GUI dashboard. The loss curve shows a much sharper, strictly decreasing trajectory with minimal variance, stabilizing effectively near 0.15. The accuracy curve demonstrates a rapid spatial feature acquisition in the early epochs, eventually plateauing tightly around the high 94% range.

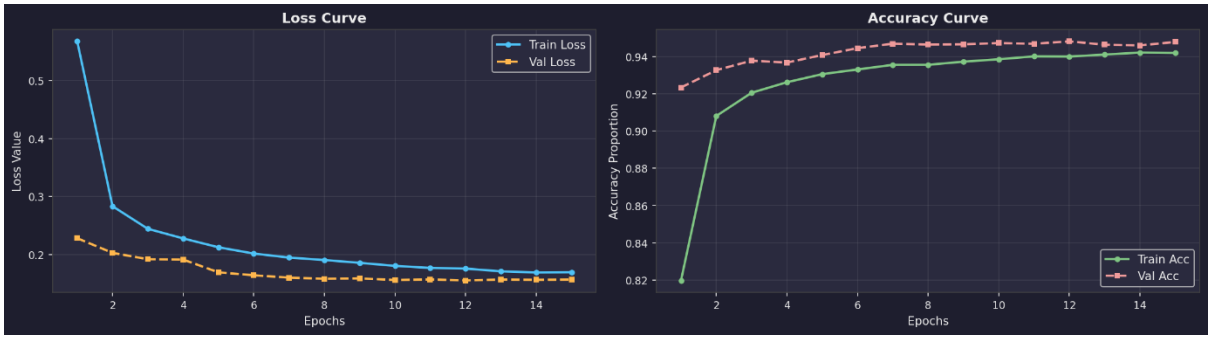

Figure 2.20. *Loss curve and accuracy curve of the CNN training result from the GUI program*

Figure 2.21 provides the detailed CNN training history from the Jupyter Notebook. It distinctly showcases the impact of the *StepLR* scheduler in the rightmost subplot, where the learning rate mathematically halves at epoch 5 and epoch 10. These step decays correspond directly with micro-stabilizations in the loss and accuracy curves, enabling the CNN to converge precisely at its optimal global minimum.

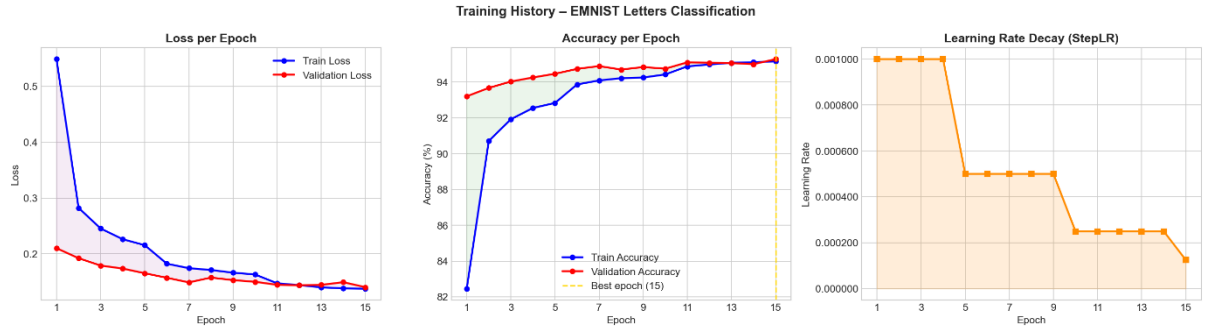

Figure 2.21. *Loss curve, accuracy curve, and learning rate decay of the CNN training result from the Jupyter Notebooks CNN*

Figure 2.22 features the interactive Per-Class Accuracy bar chart for the CNN from the GUI. With the average threshold raised to 94.8%, the overwhelming majority of the bars are marked in green (indicating >90% accuracy). The chart visually confirms that spatial convolutions drastically improved the recognition of structurally complex letters, leaving only a few problematic classes like "I" and "L" highlighted in orange.

Figure 2.22. *Per-class accuracy for CNN from the GUI program*

Figure 2.23 displays the Jupyter Notebook version of the CNN's per-class accuracy bar chart. This graph provides precise numerical labels atop each bar, confirming the exceptional performance of the model. For instance, the letter "L" scored 77.9%, and "I" scored 75.4%, which, despite being the lowest, still reflect the inherent difficulty of these specific morphological shapes in the EMNIST dataset.

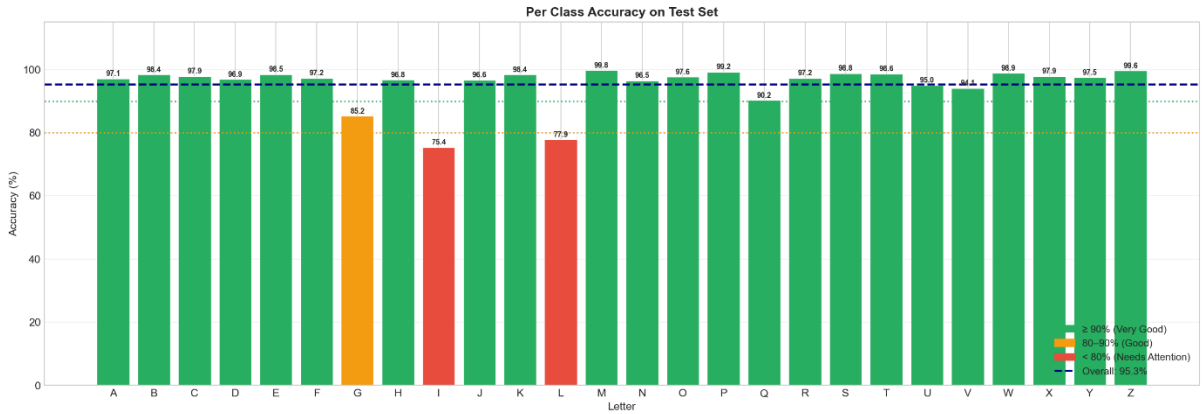

Figure 2.23. *Per-class accuracy for CNN from the Jupyter Notebooks CNN*

Figure 2.24 presents the Normalized Confusion Matrix for the CNN from the GUI. The main diagonal is noticeably darker and more sharply defined than the ANN's matrix, visually representing higher sensitivity across all classes. Off-diagonal noise is significantly diminished, proving that the CNN successfully utilized 2D local pixel relationships to distinguish previously confused characters.

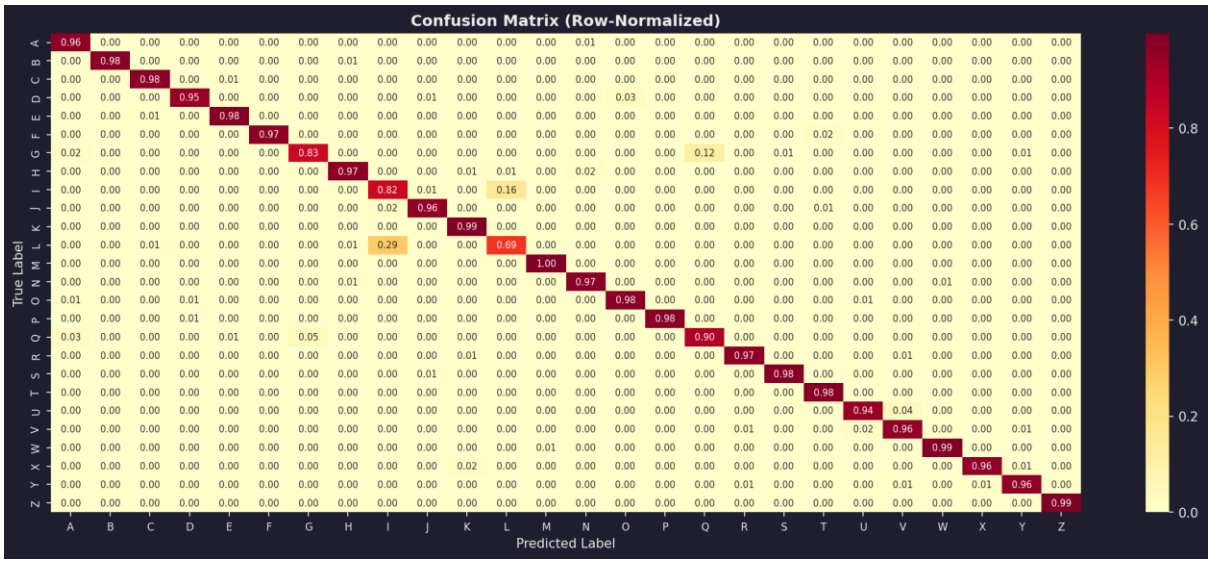

Figure 2.24. *Normalized confusion matrix of each output class of CNN from the GUI program*

Figure 2.25 shows the combined raw and normalized confusion matrices for the CNN derived from the Jupyter Notebook. The numerical annotations confirm the reduction in false positives. The intense diagonal line serves as concrete statistical evidence that the spatial feature extraction via *Conv2d* layers yields highly reliable classification boundaries.

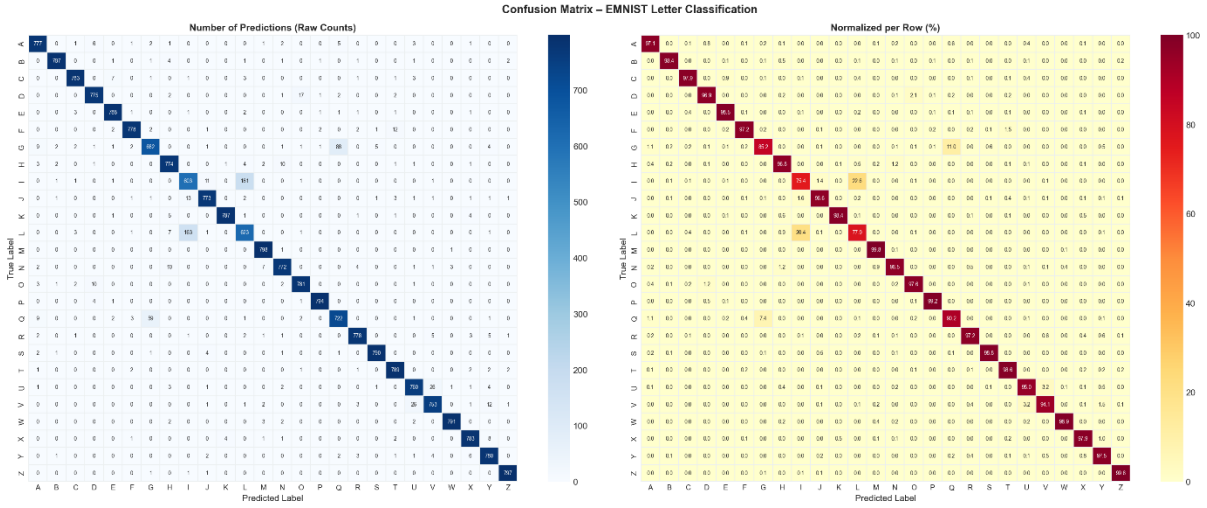

Figure 2.25. *Original confusion matrix and normalized confusion matrix of each output class of CNN from the Jupyter Notebooks CNN*

Figure 2.26 captures a highly confident correct prediction using the GUI's Prediction Explorer for the CNN. The poorly written, heavily slanted letter "A" was successfully recognized with an overwhelming 99.5% confidence probability. The probability bar chart confirms that the model decisively eliminated all other 25 classes.

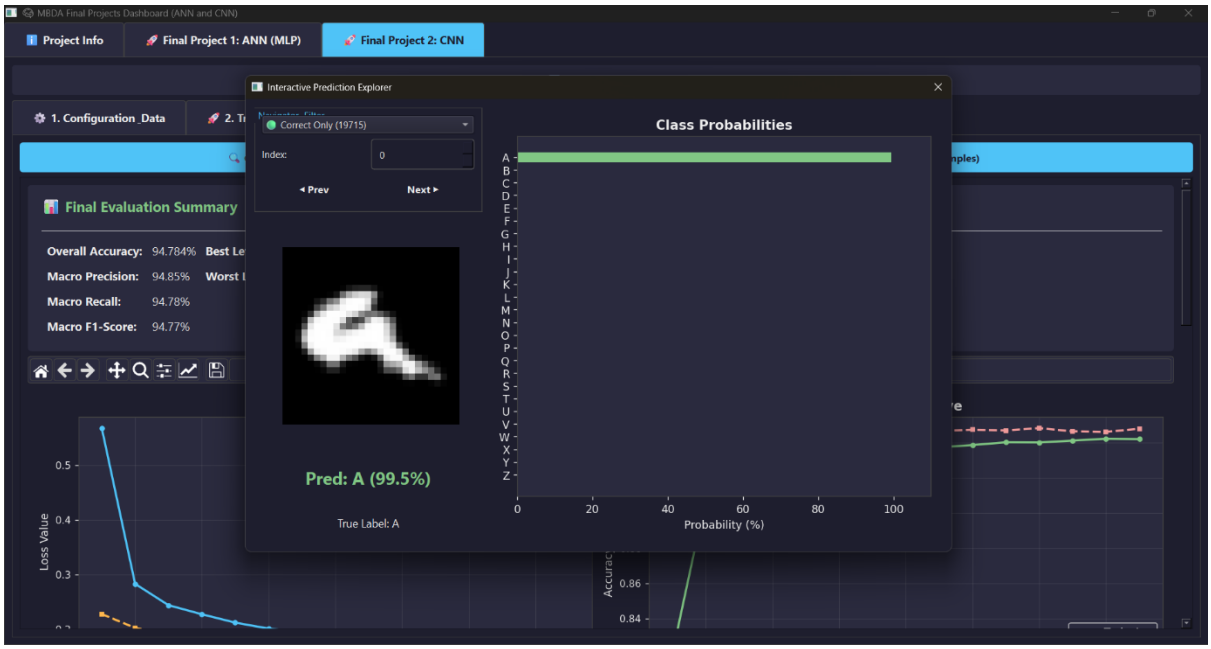

Figure 2.26. *Example of a correctly predicted class of CNN from the GUI program*

Figure 2.27 illustrates an incorrect CNN prediction from the GUI's Explorer. An ambiguous, horizontally compressed handwriting sample of an "A" was mistakenly classified as an "E" with a low confidence of 31.0%. The bar chart highlights the model's internal statistical conflict, as it struggled to extract definitive spatial edges from the heavily distorted sample.

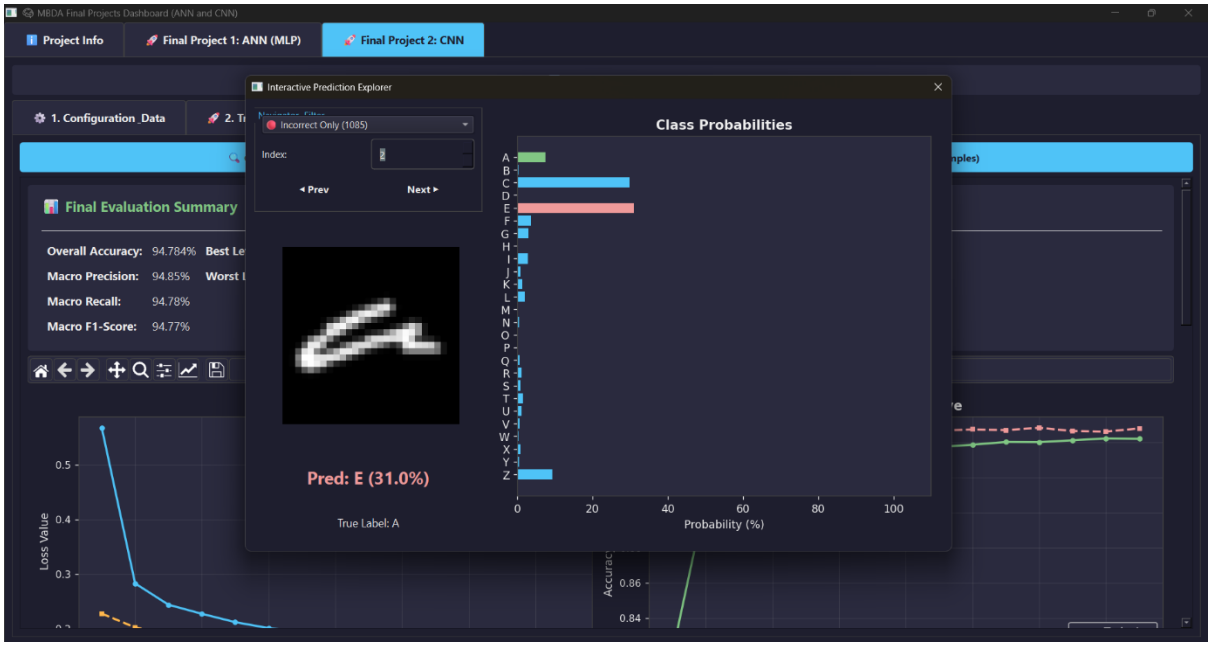

Figure 2.27. *Example of a wrongly predicted class of CNN from the GUI program*

Figure 2.28 showcases a grid of correctly classified "A" samples from the CNN's Jupyter Notebook evaluation. Despite significant variations in thickness, loops, and cursive styles, the CNN successfully identified every sample with high confidence (mostly 99% to 100%), underscoring the translation invariance provided by the Max Pooling layers.

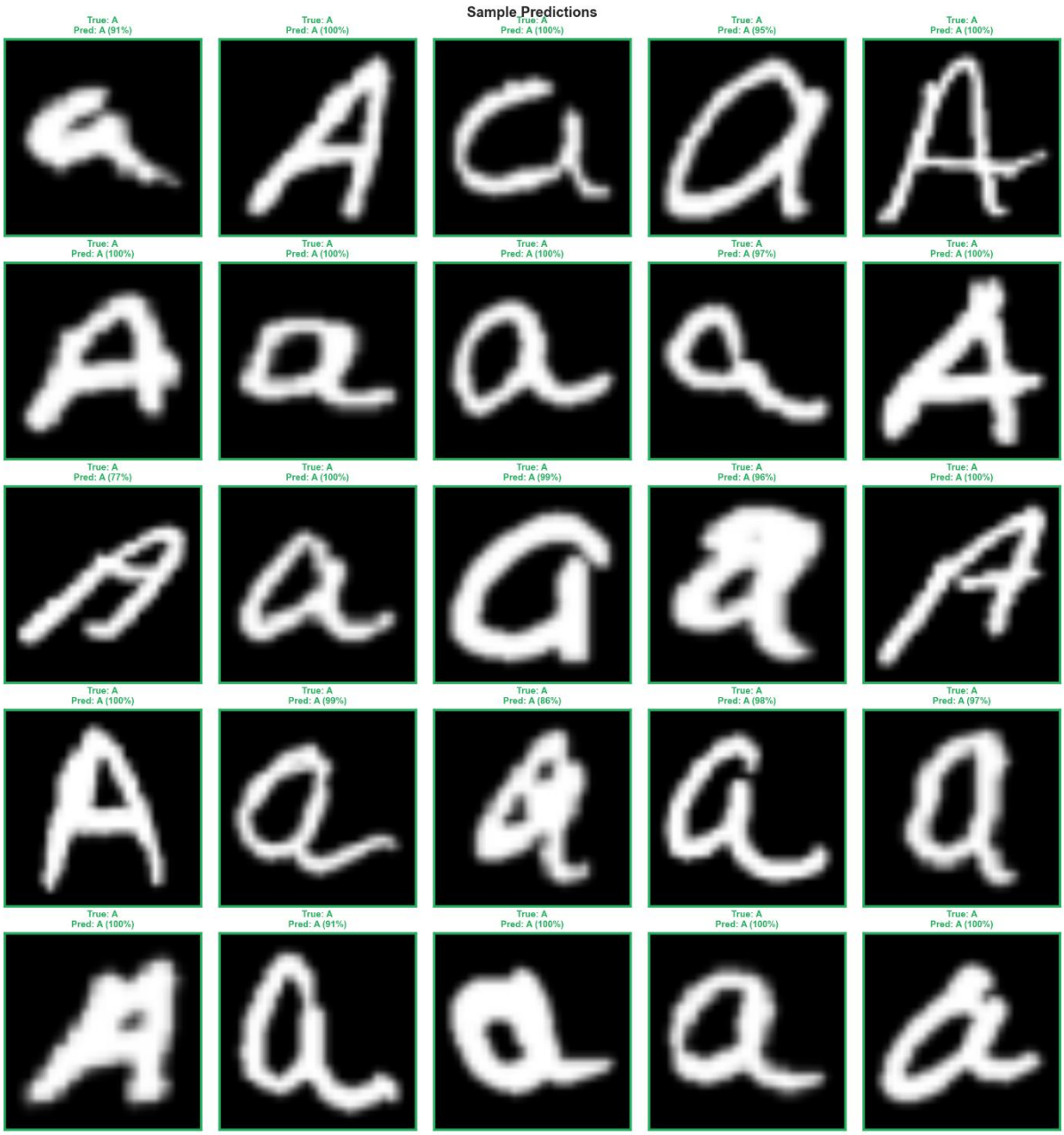

Figure 2.28. *Example of a correctly predicted class of CNN from the Jupyter Notebook CNN*

Figure 2.29 provides an Error Analysis grid from the Jupyter Notebook, exhibiting 20 wrongly predicted samples by the CNN. This visualization justifies the model's remaining failure rate, as many of these samples are highly ambiguous even to human observers (e.g., a "q" that looks identical to an "a", or a "d" lacking its vertical stem). This proves that the remaining error margin is largely attributed to inherently illegible data rather than architectural flaws.

Figure 2.29. *Example of a wronglyly predicted class of CNN from the Jupyter Notebook CNN*

2.4. Discussion and Evaluation

The comparative analysis between the ANN and the CNN reveals a definitive advantage in utilizing spatial feature extraction for complex image classification tasks. While the ANN achieved a highly respectable overall accuracy of approximately 92.6%, its fundamental architecture, which mathematically mandates flattening the 2D image arrays into 1D vectors, inherently discards critical local spatial relationships. This architectural limitation became visually evident in the per-class evaluation bar charts and confusion matrices, where the ANN struggled significantly to differentiate morphologically similar characters such as "I" and "L". In contrast, the CNN, which attained a superior overall accuracy of roughly 94.8%, successfully preserved the 2D topology of the handwritten inputs. By employing *nn.Conv2d* and *nn.MaxPool2d* layers, the CNN demonstrated a robust capability to extract hierarchical structural features (like overlapping curves and intersecting lines) while maintaining translation invariance. This spatial awareness drastically reduced false-positive misclassifications across ambiguous classes, thereby confirming the theoretical superiority of convolutional architectures in computer vision domains.

Despite the distinct architectural advantages of the CNN, the evaluation also highlighted the inherent limitations posed by the EMNIST Letters dataset itself. The *Interactive Prediction Explorer* within the GUI and the error analysis grids derived from the Jupyter Notebooks demonstrated that the remaining misclassifications were rarely due to catastrophic algorithmic failures. Instead, they predominantly stemmed from severe handwriting distortions, human input errors, and overlapping morphological traits, such as an unclosed "O" closely resembling a "U", or a heavily slanted "A" resembling an "E" as seen in **Figure 2.27**. These findings indicate that while deep learning optimization is crucial, achieving near 100% accuracy on isolated handwritten alphabets is practically bounded by the irreducible ambiguity of human handwriting. Consequently, deploying such systems in real-world Multimodal Biomedical Data Analysis pipelines, such as digitizing clinical prescriptions, would likely necessitate supplementary context-aware Natural Language Processing (NLP) models to logically infer the correct letter based on surrounding word contexts.

Beyond the raw predictive performance, this project successfully resolved the critical software engineering challenges defined in the problem statement through the development of the unified PyQt6 GUI application. By encapsulating the computationally expensive PyTorch training loops inside asynchronous *QThread* workers, the application eliminated the UI freezing issues typically associated with single-threaded machine learning software. Furthermore, the integration of native CUDA hardware acceleration, paired with memory-pinning optimizations inside the *DataLoader*, ensured that the massive matrix multiplications required by the CNN were executed rapidly and efficiently without bottlenecking the system RAM. The implementation of interactive, dynamically drawn Matplotlib dashboards directly within the application provided a highly transparent, user-friendly interface for monitoring real-time *CrossEntropyLoss* convergence and diagnosing class-specific anomalies. Ultimately, this modular architecture successfully bridges the gap between experimental data science in Jupyter Notebooks and deployable, production-ready software suitable for modern biomedical informatics environments.

CHAPTER III. CONCLUSION

This project successfully developed, evaluated, and integrated two distinct deep learning architectures, ANN and CNN, for the classification of the EMNIST Letters dataset. The comparative analysis conclusively demonstrated the superiority of convolutional architectures in computer vision tasks. While the ANN achieved a commendable overall test accuracy of approximately 92.6%, its mathematical reliance on flattening 2D image matrices into 1D vectors fundamentally restricted its ability to comprehend local structural context. Conversely, the CNN, leveraging spatial convolutions and max-pooling layers, preserved the 2D topology of the handwritten characters and achieved a higher overall accuracy of 94.8%. By maintaining translation invariance and extracting hierarchical spatial features, the CNN effectively minimized false-positive misclassifications among morphologically similar letters, proving its robustness over standard dense layers.

Despite the algorithmic success of the CNN, the granular error analysis revealed that the remaining margin of misclassification is primarily bound by the irreducible ambiguity of human handwriting. Extreme distortions, unclosed loops, and structurally identical characters, such as the lowercase "l" and uppercase "I" inherently confuse visual classifiers regardless of network depth. These dataset limitations suggest that in real-world Multimodal Biomedical Data Analysis applications, such as the digitization of handwritten clinical notes, patient intake forms, or medical prescriptions, purely visual OCR models must be augmented with contextual NLP systems to accurately infer ambiguous characters based on surrounding linguistic context.

Beyond predictive modeling, this project excelled in bridging the gap between experimental data science and deployable software engineering. The development of the unified PyQt6 GUI provided a highly interactive and transparent environment for real-time model training and evaluation. By cleanly isolating the mathematically intensive PyTorch operations and CUDA hardware acceleration within asynchronous background threads (*QThread*), the application maintained a fluid, responsive user experience without system freezes. The integration of dynamic Matplotlib dashboards, real-time terminal logging, and interactive prediction viewers ultimately proves that complex deep learning pipelines can be successfully encapsulated into robust, user-friendly desktop applications, setting a strong architectural foundation for future developments in biomedical informatics systems.

REFERENCES

[1] G. Cohen, S. Afshar, J. Tapson, and A. van Schaik, "EMNIST: Extending MNIST to handwritten letters," 2017 International Joint Conference on Neural Networks (IJCNN), Anchorage, AK, USA, 2017, pp. 2921-2926.

[2] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[3] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, no. 7553, pp. 436-444, May 2015.

[4] N. F. Hikmah, "EDA and Performance Evaluation," Lecture Notes, Biomedical Engineering Department, Institut Teknologi Sepuluh Nopember, Surabaya, 2026.

[5] N. F. Hikmah, "Convolutional Neural Network," Lecture Notes, Biomedical Engineering Department, Institut Teknologi Sepuluh Nopember, Surabaya, 2026.

[6] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Advances in Neural Information Processing Systems 32, 2019, pp. 8024-8035.

[7] M. Summerfield, Rapid GUI Programming with Python and Qt: The Definitive Guide to PyQt Programming. Upper Saddle River, NJ, USA: Prentice Hall, 2007.

[8] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," Information Processing & Management, vol. 45, no. 4, pp. 427-437, 2009.